GCA: Global Centroid Alignment in Federated Learning

Jong-Ik Park[1], **Harry Jiang**[1], **Logan Blakely**[2], **Georgios Fragkos**[2], **Shamina Hossain-McKenzie**[2], **Carlee Joe-Wong**[1]

[1]Carnegie Mellon University
[2]Sandia National Laboratories

Abstract

Autoencoder (AE)-based federated learning (FL) is attractive for anomaly detection when clients have limited local data. However, conventional FL exchanges AE parameters or gradients, incurring substantial communication overhead and potentially exposing input training data information, since AEs are explicitly optimized to reconstruct their inputs. We introduce *Global Centroid Alignment (GCA)*, a latent-code-mediated FL protocol that coordinates clients without transmitting AE parameters or gradients. In each round, (1) clients first train their local AEs using a *reconstruction* update and upload a small subset of encoder latent codes to the FL server. (2) The server pools these codes, fits a clustering model, and broadcasts only *global latent centroids and their support counts*. (3) Each client then updates its encoder by aligning its local latent codes with the *nearest* centroid using *inverse-count* weighting to emphasize globally underrepresented patterns. Steps (1)–(3) repeat over communication rounds. Because GCA exchanges only sampled latent codes and centroid statistics, its communication cost depends on latent dimensionality and the numbers of uploaded codes and returned centroids rather than on AE model size. Across five tabular and two vision benchmarks, GCA yields higher reconstruction error under a server-side client data extraction attack in all 21 comparisons and clearly lower cosine similarity in 20 of 21 comparisons with FedAvg, FedProx, and FedNova, showing its ability to protect training data. It even improves test accuracy over FedAvg by up to 5.76%. GCA achieves extraction defense comparable to DP-FedAvg, remains effective when DP-FedAvg does not reduce target resemblance, and lowers per-round communication by up to 99.15%.

Introduction

Federated learning (FL) enables multiple clients to train their own *globally informed* models *without moving their data* to another location (McMahan et al. 2017; Li et al. 2020b; Park and Joe-Wong 2024). Each client updates its model locally and sends an update (e.g., gradients or parameter changes) to a coordinating server, which aggregates these updates to form a global model and broadcasts the global model back; clients then continue local adaptation (Konečnỳ et al. 2016). This process repeats until convergence (or for a fixed number of rounds) (Li et al. 2020a; Zhang et al. 2021). Because raw data never leaves client devices, FL reduces data movement compared to centralized training and limits exposure of sensitive records, improving privacy by design (Li et al. 2021; Thapa, Chamikara, and Camtepe 2021). As a result, it is widely deployed in healthcare, finance, mobile keyboards, and industrial IoT, and is especially valuable in *data-scarce* and *data-sensitive* settings where cross-client structure can be exploited without sharing raw data (Long et al. 2020; Rieke et al. 2020).

A common task in these applications is that of detecting anomalous data, e.g., device telemetry, medical monitoring, predicting faults in industrial IoT settings, and fraud/abuse detection—where the learning task is to detect rare yet high-impact anomalies with low false-alarm rates (Kea, Han, and Kim 2023; Novoa-Paradela, Fontenla-Romero, and Guijarro-Berdiñas 2023; Shrestha et al. 2024). For such tasks, FL commonly employs reconstruction models such as *autoencoders (AEs)* (Jiang, Zhou, and Grossklags 2022; Vucovich et al. 2023). To do so, each client trains an AE on its own data (which is assumed to be normal, i.e., non-anomalous) and, in standard practice, uploads parameters or gradients of each locally trained AE model to a central server for aggregation. However, AE-based parameter sharing introduces **communication** and **unique privacy challenges**.

For **communication,** repeatedly *uploading full AE parameters is costly* for low-spec devices (e.g., microcontrollers with limited RAM and sub-Mbps uplinks) and for edge deployments (e.g., gateways with narrow, skewed traffic) (Singh et al. 2019; Gao et al. 2020). On **privacy,** because an AE is optimized to *reproduce its inputs*, its parameters (and even gradients) can encode fine-grained signatures of local samples; a curious or compromised server may exploit these signals to approximate training records (Suganuma, Ozay, and Okatani 2018; Chen and Guo 2023; Ghoshal et al. 2025). Moreover, the encoder–decoder model often collapses onto a narrow manifold that closely interpolates the training set; as a result, decoding arbitrary latent vectors—or even passing random inputs through the AE—tends to yield outputs resembling the training data (Steck 2020). The risk is amplified when a client has few examples (e.g., a small clinic with only a few hundred normal patient records per week), where overfitting makes the autoencoder mapping highly specific to the observed data (Chen et al. 2017).

We therefore propose **Global Centroid Alignment (GCA)**, an FL protocol for AE-based anomaly detection that (i) never shares raw data, parameters, or gradients; (ii) transfers sampled latent codes and centroid statistics; and (iii)

remains stable on normals-only data. **Reconstruction.** GCA begins with a local training: clients train as in standard AE learning, jointly updating the encoder and decoder so that the decoder's output closely matches the input (Tschannen, Bachem, and Lucic 2018). Clients then transmit only a subset of *latent codes* from their training data (not parameters or raw samples). **Aggregation.** The server performs *aggregation* by fitting a lightweight clustering model (e.g., a Gaussian mixture (McLachlan and Rathnayake 2014)) on the pooled latents, and broadcasts only the resulting *global latent centroids* and their *support counts*, not parameters or gradients, to the clients. **Alignment.** Finally, on the client side, an *alignment* step pulls encoder outputs (latent codes) toward the *nearest* latent centroid using an *inverse-count* weight, which emphasizes globally underrepresented patterns and discourages collapse to dominant modes (Anand et al. 2010; Mohammed, Rawashdeh, and Abdullah 2020).

Our key **contributions** in this paper consist of:

- **GCA.** A communication-efficient, latent-code-mediated FL method in which clients and a server exchange sampled latent codes, global centroids, and support counts instead of model parameters or gradients.
- **Empirical evaluation.** Across five tabular and two vision benchmarks, GCA achieves up to 5.76% higher test accuracy than FedAvg and comparable accuracy to FedProx (Li et al. 2020b), FedNova (Wang et al. 2020), and DP-FedAvg (Abadi et al. 2016; McMahan et al. 2018). GCA also provides stronger extraction defense than FedAvg, FedProx, and FedNova, and more consistent defense than DP-FedAvg, which exhibits failure cases in certain settings, while reducing per-round communication by 84.14%–99.15%.
- **Theoretical analysis.** We analyze the round-to-round stability of GCA's reconstruction and alignment updates and show that, under our threat model, the server's latent-only observations do not uniquely determine the training records.

In the following sections, the *Related Work* reviews prior studies, the *Methodology* introduces GCA, and the *Adversarial Attack Scenarios* describes the considered threat models. The *Experimental Evaluation* presents the experimental results and compares GCA's communication efficiency with standard FL methods. Finally, the *Conclusion* summarizes our findings and outlines directions for future work.

Related Work

An autoencoder consists of an encoder $E : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ and a decoder $D : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$ trained to reconstruct inputs $\mathbf{x}$ by minimizing a reconstruction loss on *normal* data (Jiang, Zhou, and Grossklags 2022; Vucovich et al. 2023). After training, a sample is scored by its reconstruction error $s(\mathbf{x}) = \|\mathbf{x} - D(E(\mathbf{x}))\|^2$, with the assumption that anomalies are poorly reconstructed and thus receive higher scores. In practice, anomaly decisions are often made by thresholding using a percentile of the *training* (or validation) error distribution to control false positives without labels. Common choices such as the 75th or 95th percentiles provide conservative cutoffs under class imbalance and concept drift (Ibidunmoye, Rezaie, and Elmroth 2017; Van Hespen et al. 2021; Kiet, Loi, and Duy 2025). Recent work also suggests that anomaly detection can benefit from *explicitly structuring* the latent space beyond reconstruction alone (e.g., encouraging normal embeddings to concentrate in a compact region (Zong et al. 2018; Zhou et al. 2021)). This paradigm is attractive when data and labels are scarce or unavailable, and has been applied broadly to tabular, time-series, and vision data (Cheng et al. 2021; Ma et al. 2025; Khowaja et al. 2025).

To collaborate without sharing raw data, clients using AEs can adopt FL with trusted partners while keeping data local, thereby avoiding the costs and risks of centralizing all records (Kea, Han, and Kim 2023; Novoa-Paradela, Fontenla-Romero, and Guijarro-Berdiñas 2023; Shrestha et al. 2024). In conventional (e.g., FedAvg-style (McMahan et al. 2017; Li et al. 2020b)) FL, clients train local AEs and periodically share model parameters or gradients for aggregation. However, sharing AE parameters or gradients can incur substantial communication overhead (Singh et al. 2019; Gao et al. 2020) and still risks exposing private information (Suganuma, Ozay, and Okatani 2018; Steck 2020; Chen and Guo 2023; Ghoshal et al. 2025).

Differentially private FL limits the influence of individual training records by clipping and perturbing model updates, but it does not directly remove an AE's underlying objective of reconstructing its inputs (Abadi et al. 2016; McMahan et al. 2018). *GCA instead avoids sharing even perturbed reconstructive AE parameters or gradients.* GCA may also appear similar to federated knowledge-sharing methods that exchange class-conditioned logits, soft labels, or feature prototypes (e.g., FedProto) (Sattler et al. 2020; Tan et al. 2022). These methods rely on predefined class semantics, whereas *GCA operates on unlabeled normal data, discovers global latent groups from uploaded latent codes,* and aligns local representations with the resulting centroids. Appendix B and Table 2 elaborate GCA's distinction from class-prototype FL and explain why withholding an AE reconstruction map as in GCA is complementary to, rather than a replacement for, differential privacy.

Methodology

In this section, we propose a new federated mechanism, **Global Centroid Alignment (GCA)**, that (i) injects *cross-client* information into each encoder *without sharing raw data, model parameters, or gradients*, (ii) respects privacy requirements and the lack of anomalous training data, and (iii) remains lightweight and stable. Figure 1 summarizes the workflow, and Appendix A (Algorithm 1) formalizes a complete round.

Setup. We consider C clients (sites with local data), indexed by $i \in \{1, \dots, C\}$, each holding *only local normal* data $\mathcal{D}_i = \{\mathbf{x}_n^{(i)} \in \mathbb{R}^{d_x}\}_{n=1}^{N_i}$. At round $r \in \{1, \dots, R\}$, the server samples a subset of participating clients $\mathcal{S}_r \subseteq \{1, \dots, C\}$ with $|\mathcal{S}_r| = m_r$. Only clients in $\mathcal{S}_r$ perform local updates and communicate with the server in round r (*line 3* in Algorithm 1). We denote the participation rate by $\phi := m_r/C$ (constant for simplicity, but can vary by round). Training runs for R global rounds; in each round, client i performs E local epochs on minibatches $B_i = \{\mathbf{x}_b\}_{b=1}^{B}$. For a sample $\mathbf{x}_b$, we define client i's encoder and decoder

$$\mathbf{z}_b \triangleq E_i(\mathbf{x}_b) \in \mathbb{R}^{d_z},\ \hat{x}_b \triangleq D_i(\mathbf{z}_b) \in \mathbb{R}^{d_x}\ (d_z \ll d_x). \quad (1)$$

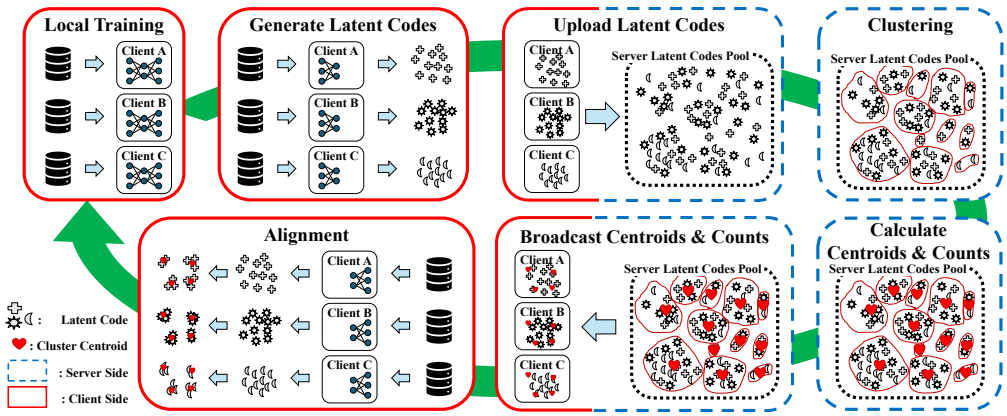

Figure 1: **GCA pipeline.** (1) *Local training:* each client trains an autoencoder on local normal data. (2) *Generate/Upload Latent upload:* clients generate and upload a subset of latent codes. (3) *Server aggregation:* the server pools latents, performs clustering, and computes global latent centroids and support counts. (4) *Broadcast:* the server sends centroids and support counts to participating clients. (5) *Alignment:* clients align their encoders to the global latent centroids using the broadcast statistics, improving cross-client consistency while keeping data and model updates private.

Each global round consists of three phases and repeats until convergence or a target metric is achieved. We allocate the total local epochs T between the reconstruction and alignment phases, $T = T_{\text{recon}} + T_{\text{align}}$. In practice, one can emphasize local stabilization early and cross-client alignment later (e.g., start with larger T_{recon} and gradually increase T_{align}), but any pair satisfying the sum constraint is admissible. In the below, all line numbers refer to Algorithm 1.

Phase 1: Reconstruction (Client-side, Encoder+Decoder)

For T_{recon} epochs, client i minimizes

$$\mathcal{L}^{(i)}_{\text{recon}}(\theta_{E_i}, \theta_{D_i}; B_i) = \frac{1}{B}\sum_{b=1}^{B} \left\|\mathbf{x}_b - D_i(E_i(\mathbf{x}_b))\right\|_2^2, \quad (2)$$

with joint updates

$$\theta_{E_i} \leftarrow \theta_{E_i} - \eta_{\text{recon}} \nabla_{\theta_{E_i}} \mathcal{L}^{(i)}_{\text{recon}},$$
$$\theta_{D_i} \leftarrow \theta_{D_i} - \eta_{\text{recon}} \nabla_{\theta_{D_i}} \mathcal{L}^{(i)}_{\text{recon}} \qquad \textit{(lines 6-13)}.$$

Starting with local reconstruction lets each AE *stabilize* on its own manifold before any global pressure is applied. After the local training, each participating client $i \in \mathcal{S}_r$ uploads only a *subset* of latent codes

$$\mathbf{z}_i^{\uparrow} \subset \{E_i(\mathbf{x}) : \mathbf{x} \in \mathcal{D}_i\}, \quad |\mathbf{z}_i^{\uparrow}| = \rho_i N_i, \quad \rho_i \in (0, 1],$$

to the server, *not* raw data and *not* model parameters or gradients *(lines 15-19)*.

Phase 2 — Aggregation (Server-side, on Pooled Latents)

After Phase 1 in round r, the server pools the uploaded latents in round r as $Z^{(r)} = \bigcup_{i\in\mathcal{S}_r} \mathbf{z}_i^{\uparrow} = \{\mathbf{z}_n\}_{n=1}^{N_r}$, where $N_r = \sum_{i\in\mathcal{S}_r} |\mathbf{z}_i^{\uparrow}|$, and fits a clustering model to Z (e.g., hard/soft K-means or a K-component Gaussian mixture model (GMM)) *(lines 21-24)*.

Example: GMM Component Means as Global Centroids. For a GMM with mixture weights $\{\pi_k\}_{k=1}^K$, means $\{\mu_k\}_{k=1}^K$, and covariances $\{\Sigma_k\}_{k=1}^K$, the expectation–maximization (EM) responsibilities, effective counts, and the component mean are $\gamma_{nk} = \frac{\pi_k \mathcal{N}(\mathbf{z}_n|\mu_k,\Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{z}_n|\mu_j,\Sigma_j)}$, $N_k = \sum_{n=1}^{N_r} \gamma_{nk}$, and $\mu_k = \frac{1}{N_k}\sum_{n=1}^{N_r} \gamma_{nk}\, \mathbf{z}_n$ respectively. In this GMM example, N_k is a soft count because it sums responsibilities rather than hard assignments. Other clustering methods similarly yield latent centroids $\{\mu_k\}$ and associated counts. The server then broadcasts the *global latent centroid set* $\mathcal{M} = \{\mu_k\}_{k=1}^K$ and *counts* $\mathbf{N}_{\mathcal{M}} = \{N_k\}_{k=1}^K$ to each participating client $i \in \mathcal{S}_r$.

Working with pooled *latent codes* avoids parameter and gradient exchange and reduces uplink cost, while clustering distills cross-client structure into a compact set of global latent centroids that clients can use without accessing other clients' data or models. The implementation broadcasts all K returned centroids and their support counts to the clients.

Phase 3 — Alignment (Client-side, Encoder-only)

For T_{align} epochs, each client i nudges its encoder toward the broadcast latent centroids while *emphasizing rare global patterns*. For each $\mathbf{x}_b$,

$$a_b \in \operatorname*{argmin}_{k\in\{1,\dots,K\}} \|E_i(\mathbf{x}_b) - \mu_k\|_2, \quad (3)$$

and we form an *inverse-count* weight with approximately *mean one* across centroids to keep the loss scale stable *(lines 27-28)*. Let $\widehat{N}_k \triangleq \max(N_k, \epsilon)$, $\bar{N} \triangleq \frac{1}{K}\sum_{k=1}^K \widehat{N}_k$, and define

$$\tilde{w}_k = \min\left\{w_{\max}, \left(\frac{\bar{N}}{\widehat{N}_k}\right)^{\gamma}\right\},$$
$$w_k = \frac{\tilde{w}_k}{\frac{1}{K}\sum_{j=1}^K \tilde{w}_j + \epsilon}. \quad (4)$$

We use hyperparameter values $\gamma = 1.0$, $w_{\max} = 6.0$, and $\epsilon = 10^{-6}$. These form a centroid-weight vector $\mathbf{w} \triangleq$

$(w_1, \ldots, w_K)^\top \in \mathbb{R}^K_{>0}$. We use w_{a_b} to index the weight of the nearest centroid for $\mathbf{x}_b$. The alignment loss is

$$\mathcal{L}^{(i)}_{\text{align}}(\theta_{E_i}; \mathcal{M}, \mathbf{w}, B_i) = \frac{1}{B} \sum_{b=1}^{B} w_{a_b} \left\| E_i(\mathbf{x}_b) - \mu_{a_b} \right\|_2^2, \quad (5)$$

and we update *only the encoder*:

$$\theta_{E_i} \leftarrow \theta_{E_i} - \eta_{\text{align}} \nabla_{\theta_{E_i}} \mathcal{L}^{(i)}_{\text{align}},$$

treating (μ_k, N_k) as constants *(lines 29-36)*.

Inference and Thresholding

For client i, the anomaly score is the reconstruction error

$$s(\mathbf{x}) \triangleq \left\| \mathbf{x} - D_i(E_i(\mathbf{x})) \right\|_2^2. \quad (6)$$

We classify an input as normal if $s(\mathbf{x}) \leq \tau_i$ and anomalous otherwise. Since anomaly labels are unavailable during training, we set τ_i using a *percentile* threshold computed from client i's training errors:

$$\tau_i = \text{Quantile}_{p_i}\left(\{ s(\mathbf{x}) : \mathbf{x} \in \mathcal{D}_i^{\text{train}} \} \right), \quad p_i \in [0, 1]. \quad (7)$$

Remark. The reconstruction, aggregation, and alignment schedule first allows each client to learn a stable local manifold, then injects cross-client structure through global latent centroids, reducing overfitting to site-specific idiosyncrasies. Inverse-count weighting in the alignment step further improves coverage of rare or under-represented modes by upweighting centroids with low global support (Anand et al. 2010; Chang et al. 2013; Li, Kamnitsas, and Glocker 2020). Because clients upload only a fraction of their latent codes and the server returns a compact set of centroids and counts, per-round communication depends on the uploaded-code count, latent dimension, and number of centroids rather than model size; no parameters or gradients are exchanged (Yamazaki et al. 2022; Jia et al. 2024).

We also provide theoretical support for our design in Appendix C, establishing (1) stable convergence to a minimizer in the convex case and (2) finite-time convergence to a critical point in the non-convex case.

Adversarial Attack Scenarios

This section formalizes the server-side threat models considered for *autoencoder-based* federated anomaly detection and our GCA protocol. We focus on *training data extraction* attacks, since autoencoders are trained to reproduce normal inputs and may therefore leak input training data information through their reconstructions. We consider a malicious (or honest-but-curious) server under two settings: (i) **standard FL**, where the server receives locally trained AE models (parameters or gradients), and (ii) **GCA**, where the server receives only uploaded latent codes, and never receives parameters or gradients. Figure 2 depicts the standard-FL inversion and GCA latent-only workflows. In both cases, the adversary's goal is to reconstruct or closely approximate the private training records of a victim client.

Standard FL: DeepDream-style Model Inversion

DeepDream-style attacks synthesize an input by *directly optimizing the input* against a trained network, following (Mordvintsev, Olah, and Tyka 2015; Ghoshal et al. 2025). First, we consider a generic classifier with logits $s(\mathbf{x}) \in \mathbb{R}^K$ and a target class y. A canonical DeepDream/feature-visualization objective performs gradient ascent on the class score:

$$\max_{\mathbf{x} \in \mathcal{X}} \; s_y(\mathbf{x}) \quad \Longrightarrow \quad \mathbf{x}^{(t+1)} = \Pi_{\mathcal{X}}\Big(\mathbf{x}^{(t)} + \eta \, \nabla_{\mathbf{x}} s_y(\mathbf{x}^{(t)}) \Big), \quad (8)$$

where $\eta > 0$ is a step size and $\Pi_{\mathcal{X}}$ projects onto the valid input domain $\mathcal{X}$ (e.g., box constraints such as pixel bounds).

The same input-optimization template applies to autoencoders once we replace the classification score with a reconstruction-based objective. For a victim client i, let the autoencoder define an encoder $E_i : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ and decoder $D_i : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, with reconstruction map

$$T_i(\mathbf{x}) \; \triangleq \; D_i(E_i(\mathbf{x})). \quad (9)$$

A natural "DeepDream-style" objective for an AE is to find inputs that the AE reconstructs *especially well*, i.e., inputs with small reconstruction error $s_i(\mathbf{x})$ (the same anomaly score used by client i in GCA's inference stage):

$$\min_{\mathbf{x} \in \mathcal{X}} \; s_i(\mathbf{x}), \qquad s_i(\mathbf{x}) \; \triangleq \; \|\mathbf{x} - T_i(\mathbf{x})\|_2^2. \quad (10)$$

This mirrors (8) but uses the reconstruction energy instead of a class logit.

In standard FL, the server receives client-side AE models (parameters or gradients). Thus, for a targeted victim client i, the server can obtain a *white-box* copy of (E_i, D_i) and compute $\nabla_{\mathbf{x}} s_i(\mathbf{x})$ by backpropagation. Starting from an initialization $\mathbf{x}^{(0)}$ (e.g., random noise or a public sample), the malicious server performs gradient descent on (10):

$$\mathbf{x}^{(t+1)} \; = \; \Pi_{\mathcal{X}}\Big(\mathbf{x}^{(t)} - \eta \nabla_{\mathbf{x}} s_i\big(\mathbf{x}^{(t)}\big) \Big), \quad t = 0, 1, \ldots, T-1, \quad (11)$$

and outputs $\tilde{\mathbf{x}} = T_i(\mathbf{x}^{(T)})$ as an extracted training-like record.

Remark. Autoencoders can be particularly vulnerable to this style of inversion because their training objective explicitly encourages *input reproduction* rather than correct classification. We provide a theoretical discussion in Appendix D.

GCA: Latent Code Leakage via Decoder–Encoder "Flipping"

In GCA, the server never observes the client's AE parameters or gradients; instead, the server receives only the latent codes. For a victim client i, let

$$Z_i^{\uparrow} = \{ \mathbf{z}_n^{(i)} \}_{n=1}^{M_i} \subset \mathbb{R}^{d_z}, \qquad \mathbf{z}_n^{(i)} = E_i(\mathbf{x}_n^{(i)}), \quad (12)$$

denote the set of uploaded latents (with $M_i = \rho_i N_i$). Although the server does not have access to the victim decoder D_i, it may still attempt to recover training-like inputs by *learning a surrogate inverse map* from latent space back to input space using only the received latent samples.

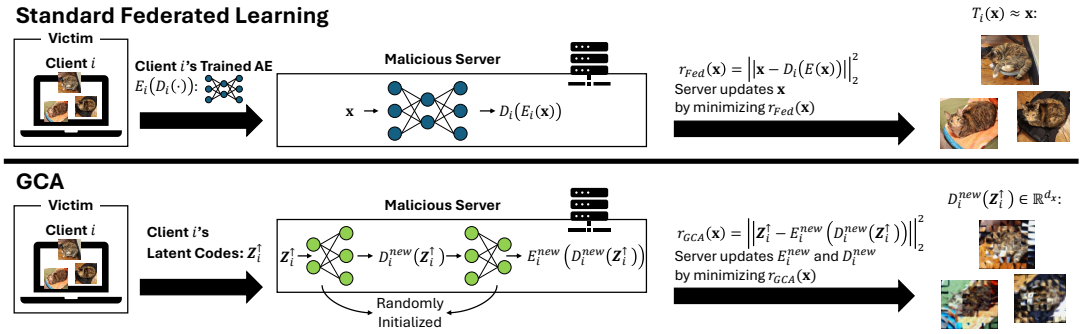

Figure 2: **Server-side training-data extraction threat models. Top: Standard FL.** The server obtains the victim reconstruction map and performs DeepDream-style input optimization. **Bottom: GCA.** The server receives uploaded latent codes, trains a fresh surrogate reconstruction map, and applies the same input optimization. Both attacks output the model reconstruction of the optimized input.

Decoder–encoder flipping attack. The server constructs a *surrogate* autoencoder whose *input* is a latent code $\mathbf{z} \in \mathbb{R}^{d_z}$ and whose *output* lies in the original input space $\mathbb{R}^{d_x}$. Concretely, the server defines "flipped" encoder and decoder $\tilde{D}_i : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and $\tilde{E}_i : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, and trains the pair $(\tilde{D}_i, \tilde{E}_i)$ *only on client i's uploaded codes* (i.e., without mixing latents across clients) by minimizing a latent-space reconstruction objective

$$\min_{\theta_{\tilde{D}_i}, \theta_{\tilde{E}_i}} \frac{1}{M_i} \sum_{n=1}^{M_i} \|\mathbf{z}_n^{(i)} - \hat{\mathbf{z}}_n^{(i)}\|_2^2, \quad \hat{\mathbf{z}}_n^{(i)} \triangleq \tilde{E}_i\big(\tilde{D}_i(\mathbf{z}_n^{(i)})\big). \quad (13)$$

This procedure is supervised in the sense that each latent $\mathbf{z}_n^{(i)}$ serves as its own training target, even though no raw inputs $\mathbf{x}_n^{(i)}$ are available.

Surrogate input optimization. After latent-only training, the server defines the surrogate reconstruction map

$$\tilde{T}_i(\mathbf{x}) \triangleq \tilde{D}_i(\tilde{E}_i(\mathbf{x})) \quad (14)$$

and applies the same update in (11), replacing T_i with $\tilde{T}_i$. The GCA attack outputs $\tilde{T}_i(\mathbf{x}^{(T)})$; direct decodes $\tilde{D}_i(\mathbf{z})$ are retained only as a GCA-specific source diagnostic.

Remark. This attack leverages the fact that GCA reveals a set of latent codes $Z_i^{\uparrow}$ for each client. Unlike standard-FL inversion, it does not require backpropagation through the *victim* autoencoder. Because the server trains $(\tilde{D}_i, \tilde{E}_i)$ from scratch using only latent samples, neither $\tilde{D}_i(\mathbf{z})$ nor $\tilde{T}_i(\mathbf{x})$ is guaranteed to be semantically faithful in input space. Appendix D discusses this underdetermination.

Experimental Evaluation

Through our experimental validation, we find that GCA consistently outperforms isolated Single-Client training and achieves test accuracy comparable to parameter-sharing FL methods. For privacy, GCA provides stronger client-data extraction defense than FedAvg, FedProx, and FedNova, and more consistent defense than DP-FedAvg under the attacks discussed in *Adversarial Attack Scenarios*.We also show that GCA substantially reduces communication costs.

Anomaly Detection Evaluation

Setup. We evaluate GCA against *Single-Client* (no collaboration), *Centralized* (pooled data), *FedAvg* (sample-weighted model averaging), *FedProx* (FedAvg with proximal regularization), *FedNova* (normalized updates), and *DP-FedAvg* (differential-privacy-style clipped/noisy FedAvg) on five tabular and two vision datasets using three seeds. Each independent-and-identically-distributed (IID) case partitions training data into C disjoint uniform shards. Tabular models use a feed-forward AE (encoder $d_x \to 256 \to 64 \to 16$, symmetric decoder; ReLU; linear head). Vision models use a convolutional AE with flattened latent dimensions 1,024 (CIFAR-10) and 784 (Fashion-MNIST).

All methods are optimized with Adam (learning rate 10^{-3}) and trained for $R = 100$ global rounds with a mini-batch size of 50. In GCA, each round consists of $T_{\text{recon}} = 5$ local epochs of reconstruction followed by $T_{\text{align}} = 5$ local epochs of alignment (Algorithm 1). To match total local computation per round, **FedAvg**, **FedProx**, **FedNova**, **DP-FedAvg**, **Single-Client**, and **Centralized** use 10 local epochs per round/epoch-equivalent. FedAvg and FedProx aggregate full model parameters; FedProx uses a configured proximal coefficient $\mu = 10^{-2}$ and an effective implemented proximal-loss multiplier of 10^{-4}.

In our experiments, all FL variants and GCA use *full client participation* in every round, i.e., $\mathcal{S}_r = \{1, \ldots, C\}$ and $\phi = 1$. In GCA, after the reconstruction phase, each client uploads a subset of latent codes with size $|Z_i^{\uparrow}| = \rho_i N_i$. The server pools the uploaded latent codes as $Z^{(r)} = \bigcup_{i=1}^{C} Z_i^{\uparrow}$, fits a K-component GMM using a modality-specific covariance type, and broadcasts the non-empty component means and effective support counts (μ_k, N_k) to all clients. Each client then performs encoder-only alignment using inverse-count weights normalized to have approximately mean one, as described in the *Methodology* section.

Dataset-specific (client count C, selected GMM component count $K^{\star}$, latent upload fraction ρ) values are: *Predict Students' Dropout and Academic Success* $(10, 10, 0.5)$; *Adult Income* $(20, 10, 0.1)$; *MAGIC* $(10, 80, 0.25)$; *Credit Card Fraud Detection* $(50, 20, 0.1)$; *Bank Marketing*

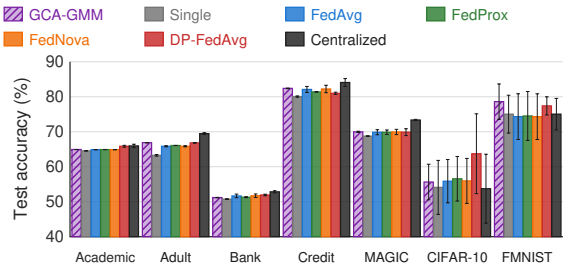

Figure 3: **IID anomaly-detection test accuracy; higher is better.** GCA outperforms Single-Client training on all seven datasets and achieves accuracy comparable to the parameter-sharing FL baselines. Bars show three-seed means and standard deviations. Full results are in Appendix E (Table 6).

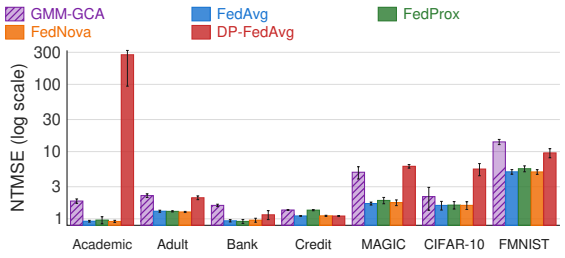

Figure 4: **Normalized target MSE (NTMSE) under the server-side extraction attack; higher is better for extraction defense.** Higher NTMSE means that the attack outputs have larger reconstruction error relative to held-out normal records and are therefore less similar to the client's training data. Bars show three-seed means and standard deviations. Full results are in Appendix E (Table 10).

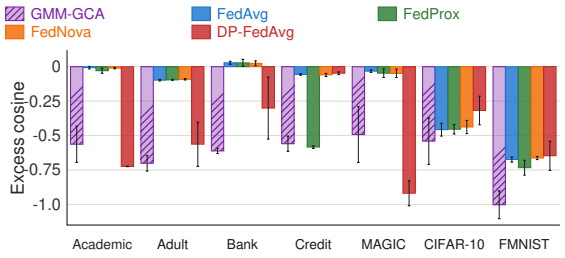

Figure 5: **Excess cosine similarity under the server-side extraction attack; lower is better for extraction defense.** Lower excess cosine similarity means that the attack outputs are less similar to the client's training data relative to held-out normal records. Bars show three-seed means and standard deviations. Full results are in Appendix E (Table 10).

(Full) $(20, 10, 0.1)$; *CIFAR-10* $(20, 20, 0.1)$; *Fashion-MNIST* $(20, 10, 0.1)$.

To fit the GMM model to the pooled latent codes, we initialize the component means using Torch K-means++ clustering. For tabular datasets, we use a Torch full-covariance GMM with covariance regularization 10^{-6} and one initialization. For vision datasets, we use a Torch diagonal-covariance GMM with covariance regularization 0.1 and 100 initializations. In all cases, we run EM for at most 200 iterations with tolerance 10^{-3}.

Training uses only "normal" data on each client (one-class setting). For *Adult*, *CIFAR-10*, and *Fashion-MNIST*, we evaluate on the provided test splits. For the remaining tabular datasets, we form a balanced test set from the anomalous rows and an equal-sized held-out tail of the normal rows. For tabular preprocessing, we remove highly correlated features (Pearson correlation > 0.6) and apply standard normalization (fit on training normals and applied to test). Appendix A documents common settings, preprocessing and partitions, GCA and every baseline, attack implementations, and reproducibility controls.

Evaluation Protocol. The baselines are evaluated once per round using their aggregated global models. Since GCA does not produce a global model, it is evaluated by averaging the local models' test accuracies during the reconstruction phase for a certain epoch. For each seed, we report the highest test accuracy across rounds. We select $K^\star$ as the number of clusters that yields the highest three-seed mean accuracy among all tested K values.

Results. Figure 3 reports the primary IID comparison. GCA outperforms Single-Client on all seven datasets and FedAvg on five of seven, with relative changes against FedAvg ranging from -0.92% to $+5.76\%$. It remains competitive with FedProx, FedNova, and DP-FedAvg, with win/loss counts of 5/2, 5/2, and 4/3, respectively. Exact per-dataset results are provided in Appendix E (Table 6).

Ablations. Appendix E supports $K^\star$ with the GMM sweep (Tables 3–4), compares inverse and uniform alignment weights (Table 5), tests three clustering backends (Tables 7–9), and reports the non-IID K sweep and baseline comparison (Tables 13 and 14).

Adversarial Evaluation

Setup. We implement the two server-side extraction attacks defined in *Adversarial Attack Scenarios*. For FedAvg, FedProx, FedNova, and DP-FedAvg, the server applies the white-box model-inversion attack to the client's final-round autoencoder. For GCA, the client starts from its final-round checkpoint and performs five additional local reconstruction epochs before uploading the configured latent sample, following GCA's standard reconstruction-and-upload procedure. An isolated server-side process then trains a fresh decoder–encoder flipping surrogate using only the latent-cycle loss in (13) and applies the same input-optimization update in (11), without access to client parameters or raw records (Appendix A).

Symmetric attack protocol. Both attacks use the same 100 starts and DeepDream optimizer: independent Adam (10^{-2}) runs of at most 1000 steps, with per-start best-state tracking and 10-step relative-improvement stopping at 10^{-3}. Vision is projected to $[-1, 1]$ and standardized tabular inputs are unbounded. The output is the reconstruction $T(\mathbf{x}^\star)$ rather

than the optimized input itself, using the client's reconstruction map for the parameter-sharing methods and the latent-only surrogate for GCA. Matching starts and optimization settings isolate the effect of the server-visible reconstruction map rather than giving either communication protocol a different search budget.

Metrics. For every training target $\mathbf{x}$, let $r^\star(\mathbf{x})$ be its MSE (mean-squared error)-nearest output among the 100 reconstructions. Cosine similarity is evaluated against this same $r^\star(\mathbf{x})$, so the attack cannot use one output for the distance result and another for the similarity result. With a reference bank of 100 normal records excluded from training, NTMSE is the normalized target MSE, defined as the mean target-to-output MSE divided by the mean target-to-held-out-record MSE. Excess cosine similarity, $\Delta\cos$, is the mean target-to-output cosine similarity minus the mean target-to-held-out-record cosine similarity. *Higher NTMSE and lower* $\Delta\cos$ *therefore mean that the attack outputs are less similar to the training shard relative to ordinary held-out records.* The normalization is performed within each dataset, client, and seed; we do not average raw distances across datasets with different feature scales. Complete definitions and all per-dataset values are reported in Appendix E and Table 10.

Results. Figures 4 and 5 summarize the two leakage measures. We first compare GCA with DP-FedAvg. GCA has higher NTMSE on four of seven datasets and lower excess cosine similarity on five of seven, favoring GCA in 9 of the 14 dataset–metric comparisons. GCA is favored by both metrics on Adult, Bank, Credit, and Fashion-MNIST, whereas DP-FedAvg has higher NTMSE on Academic, MAGIC, and CIFAR-10. More importantly, *DP-FedAvg does not consistently improve over the standard FL variants*: in 4 of the 14 comparisons—both metrics on Credit and excess cosine similarity on CIFAR-10 and Fashion-MNIST—it performs worse than FedAvg, FedProx, and FedNova. In contrast, GCA consistently maintains low target resemblance across these settings. These results indicate that GCA provides more consistent empirical extraction defense than the tested DP-FedAvg configuration.

Compared with the standard parameter-sharing FL methods, GCA has higher NTMSE than FedAvg, FedProx, and FedNova on every dataset, giving 21 of 21 favorable comparisons. It also has lower excess cosine similarity in 20 of 21 comparisons: seven of seven against FedAvg, seven of seven against FedNova, and six of seven against FedProx. The only exception is Credit against FedProx, where GCA has slightly higher NTMSE (1.352 versus 1.346) but slightly higher excess cosine similarity (-0.559 versus -0.584).

The size of the separation varies by dataset. On Academic, GCA reaches an NTMSE of 1.838 and excess cosine similarity of -0.563, compared with 0.921 and -0.007 for FedAvg. On Adult, the corresponding values are 2.216 and -0.702 for GCA versus 1.298 and -0.098 for FedAvg. GCA achieves its highest NTMSE on Fashion-MNIST (13.940), where it also obtains its lowest excess cosine similarity (-1.003). These results show that the GCA surrogate outputs remain farther from and less directionally similar to the clients' training records than the outputs obtained from the parameter-sharing FL methods.

Overall, GCA provides stronger and more consistent empirical extraction defense than FedAvg, FedProx, FedNova, and the tested DP-FedAvg configuration. Moreover, GCA reduces per-round communication by 84.14%–99.15%, providing a more favorable empirical accuracy–extraction-defense–communication trade-off in our evaluated settings (see the discussion below and Appendix E). Full results are provided in Appendix E (Table 10).

Communication Efficiency

Metrics. Let C be the number of clients, P the AE parameter count, D_z the flattened latent dimension, and $U = \sum_{i=1}^{C} L_i$ the total number of uploaded latent codes in one round. All clients participate; every scalar uses 32-bit floating point (FP32), so $\beta = 4$ bytes. GCA uploads latent codes and broadcasts K centroids and their K support counts to every client. The round-level payloads are

$$B^{\uparrow}_{\mathrm{GCA}} = \beta U D_z, \quad B^{\downarrow}_{\mathrm{GCA}} = \beta C K (D_z + 1),$$
$$B^{\uparrow}_{\mathrm{FL}} = \beta C P, \qquad B^{\downarrow}_{\mathrm{FL}} = \beta C P.$$

Thus $B_{\mathrm{GCA}} = \beta[U D_z + C K (D_z + 1)]$ and $B_{\mathrm{FL}} = 2\beta C P$ per round. In terms of scaling, FL is $\mathcal{O}(CP)$, whereas GCA is $\mathcal{O}(UD_z + CKD_z)$ and does not depend on model size. Under this accounting, GCA communicates less whenever $U < \frac{C[2P - K(D_z+1)]}{D_z}$. The upload fraction controls U, while the bottleneck architecture controls D_z.

Results. Across these evaluated settings, GCA reduces per-round communication by 84.14%–99.15% relative to full-model FL. Appendix A defines the full scope of these settings; Appendix E lists C, U, D_z, P in Table 11 and reports every dataset and $K \in \{10, 20, 40, 80\}$ in Table 12.

Conclusion

GCA replaces AE parameter and gradient exchange with sampled latent codes, global centroids, and support counts. Across seven datasets, GCA outperforms FedAvg on five datasets by up to 5.76% while reducing per-round communication by 84.14%–99.15%.

Under the matched extraction attack, GCA has higher normalized target MSE in all 21 comparisons and lower excess cosine similarity in 20 of 21 comparisons with FedAvg, FedProx, and FedNova. GCA also provides more consistent extraction defense than the tested DP-FedAvg configuration, outperforming it in 9 of 14 dataset–metric comparisons.

Future work will consider stronger adversaries, dynamic centroids, uncertainty-aware responsibilities, and robust or privacy-preserving clustering mechanisms with formal privacy guarantees.

References

Abadi, M.; Chu, A.; Goodfellow, I.; McMahan, H. B.; Mironov, I.; Talwar, K.; and Zhang, L. 2016. Deep Learning with Differential Privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 308–318. ACM.

Anand, A.; Pugalenthi, G.; Fogel, G. B.; and Suganthan, P. 2010. An approach for classification of highly imbalanced data using weighting and undersampling. *Amino Acids*, 39(5): 1385–1391.

Bolte, J.; Sabach, S.; and Teboulle, M. 2014. Proximal alternating linearized minimization for nonconvex and nonsmooth problems. *Mathematical Programming*, 146(1): 459–494.

Bottou, L.; Curtis, F. E.; and Nocedal, J. 2018. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2): 223–311.

Chang, C.-Y.; Hsu, M.-T.; Esposito, E. X.; and Tseng, Y. J. 2013. Oversampling to overcome overfitting: exploring the relationship between data set composition, molecular descriptors, and predictive modeling methods. *Journal of Chemical Information and Modeling*, 53(4): 958–971.

Chen, J.; Sathe, S.; Aggarwal, C.; and Turaga, D. 2017. Outlier detection with autoencoder ensembles. In *Proceedings of the 2017 SIAM International Conference on Data Mining*, 90–98. SIAM.

Chen, S.; and Guo, W. 2023. Auto-encoders in deep learning—a review with new perspectives. *Mathematics*, 11(8): 1777.

Cheng, Z.; Wang, S.; Zhang, P.; Wang, S.; Liu, X.; and Zhu, E. 2021. Improved autoencoder for unsupervised anomaly detection. *International Journal of Intelligent Systems*, 36(12): 7103–7125.

Frankel, P.; Garrigos, G.; and Peypouquet, J. 2014. Splitting Methods with Variable Metric for Kurdyka–Łojasiewicz Functions and General Convergence Rates. *Journal of Optimization Theory and Applications*, 165(3): 874–900.

Gao, Y.; Kim, M.; Abuadbba, S.; Kim, Y.; Thapa, C.; Kim, K.; Camtep, S. A.; Kim, H.; and Nepal, S. 2020. End-to-End Evaluation of Federated Learning and Split Learning for Internet of Things. In *2020 International Symposium on Reliable Distributed Systems (SRDS)*, 91–100. IEEE.

Ghoshal, A.; Das, R.; Kundu, A.; and De, I. 2025. Reverse-Engineering Autoencoders: Optimized Gradient-Inversion Attacks for Sensitive Data Extraction. *Authorea Preprints*.

Ibidunmoye, O.; Rezaie, A.-R.; and Elmroth, E. 2017. Adaptive anomaly detection in performance metric streams. *IEEE Transactions on Network and Service Management*, 15(1): 217–231.

Jia, Z.; Li, J.; Li, B.; Li, H.; and Lu, Y. 2024. Generative latent coding for ultra-low bitrate image compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 26088–26098.

Jiang, X.; Zhou, X.; and Grossklags, J. 2022. Privacy-preserving high-dimensional data collection with federated generative autoencoder. *Proceedings on Privacy Enhancing Technologies*.

Karimi, H.; Nutini, J.; and Schmidt, M. 2016. Linear convergence of gradient and proximal-gradient methods under the polyak-łojasiewicz condition. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, 795–811. Springer.

Kea, K.; Han, Y.; and Kim, T.-K. 2023. Enhancing anomaly detection in distributed power systems using autoencoder-based federated learning. *PLOS One*, 18(8): e0290337.

Khowaja, S. A.; Dev, K.; Anwar, S. M.; and Linguraru, M. G. 2025. SelfFed: Self-supervised federated learning for data heterogeneity and label scarcity in medical images. *Expert Systems with Applications*, 261: 125493.

Kiet, T. T.; Loi, N. T.; and Duy, V. N. L. 2025. Statistical Inference for Autoencoder-based Anomaly Detection after Representation Learning-based Domain Adaptation. *arXiv preprint arXiv:2508.07049*.

Konečnỳ, J.; McMahan, H. B.; Yu, F. X.; Richtárik, P.; Suresh, A. T.; and Bacon, D. 2016. Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492*.

Li, L.; Fan, Y.; Tse, M.; and Lin, K.-Y. 2020a. A review of applications in federated learning. *Computers & Industrial Engineering*, 149: 106854.

Li, Q.; Wen, Z.; Wu, Z.; Hu, S.; Wang, N.; Li, Y.; Liu, X.; and He, B. 2021. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering*, 35(4): 3347–3366.

Li, T.; Sahu, A. K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; and Smith, V. 2020b. Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems*, 2: 429–450.

Li, Z.; Kamnitsas, K.; and Glocker, B. 2020. Analyzing overfitting under class imbalance in neural networks for image segmentation. *IEEE Transactions on Medical Imaging*, 40(3): 1065–1077.

Long, G.; Tan, Y.; Jiang, J.; and Zhang, C. 2020. Federated learning for open banking. In *Federated Learning: Privacy and Incentive*, 240–254. Springer.

Ma, Z.; Li, Z.; Shi, Y.; and Chen, J. 2025. FedSum: Data-Efficient Federated Learning Under Data Scarcity Scenario for Text Summarization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 19340–19348.

McLachlan, G. J.; and Rathnayake, S. 2014. On the number of components in a Gaussian mixture model. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 4(5): 341–355.

McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; and y Arcas, B. A. 2017. Communication-efficient learning of deep networks from decentralized data. In *International Conference on Artificial Intelligence and Statistics*, 1273–1282. PMLR.

McMahan, H. B.; Ramage, D.; Talwar, K.; and Zhang, L. 2018. Learning Differentially Private Recurrent Language Models. In *International Conference on Learning Representations*.

Mohammed, R.; Rawashdeh, J.; and Abdullah, M. 2020. Machine learning with oversampling and undersampling techniques: overview study and experimental results. In *2020 11th International Conference on Information and Communication Systems (ICICS)*, 243–248. IEEE.

Mordvintsev, A.; Olah, C.; and Tyka, M. 2015. Inceptionism: Going deeper into neural networks. *Google research blog*, 20(14): 5.

Novoa-Paradela, D.; Fontenla-Romero, O.; and Guijarro-Berdiñas, B. 2023. Fast deep autoencoder for federated learning. *Pattern Recognition*, 143: 109805.

Park, J.-I.; and Joe-Wong, C. 2024. Federated learning with flexible architectures. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, 143–161. Springer.

Rieke, N.; Hancox, J.; Li, W.; Milletari, F.; Roth, H. R.; Albarqouni, S.; Bakas, S.; Galtier, M. N.; Landman, B. A.; Maier-Hein, K.; et al. 2020. The future of digital health with federated learning. *NPJ Digital Medicine*, 3(1).

Sattler, F.; Marban, A.; Rischke, R.; and Samek, W. 2020. Communication-Efficient Federated Distillation. *arXiv preprint arXiv:2012.00632*.

Shrestha, R.; Mohammadi, M.; Sinaei, S.; Salcines, A.; Pampliega, D.; Clemente, R.; Sanz, A. L.; Nowroozi, E.; and Lindgren, A. 2024. Anomaly detection based on lstm and autoencoders using federated learning in smart electric grid. *Journal of Parallel and Distributed Computing*, 193: 104951.

Singh, A.; Vepakomma, P.; Gupta, O.; and Raskar, R. 2019. Detailed comparison of communication efficiency of split learning and federated learning. *arXiv preprint arXiv:1909.09145*.

Steck, H. 2020. Autoencoders that don't overfit towards the identity. *Advances in Neural Information Processing Systems*, 33: 19598–19608.

Suganuma, M.; Ozay, M.; and Okatani, T. 2018. Exploiting the potential of standard convolutional autoencoders for image restoration by evolutionary search. In *International Conference on Machine Learning*. PMLR.

Tan, Y.; Long, G.; Liu, L.; Zhou, T.; Lu, Q.; Jiang, J.; and Zhang, C. 2022. FedProto: Federated Prototype Learning across Heterogeneous Clients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 8432–8440.

Thapa, C.; Chamikara, M. A. P.; and Camtepe, S. A. 2021. Advancements of federated learning towards privacy preservation: from federated learning to split learning. In *Federated Learning Systems: Towards Next-Generation AI*, 79–109. Springer.

Tschannen, M.; Bachem, O.; and Lucic, M. 2018. Recent advances in autoencoder-based representation learning. *arXiv preprint arXiv:1812.05069*.

Tseng, P. 2001. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of Optimization Theory and Applications*, 109(3): 475–494.

Van Hespen, K. M.; Zwanenburg, J. J.; Dankbaar, J. W.; Geerlings, M. I.; Hendrikse, J.; and Kuijf, H. J. 2021. An anomaly detection approach to identify chronic brain infarcts on MRI. *Scientific Reports*, 11(1): 7714.

Vucovich, M.; Tarcar, A.; Rebelo, P.; Rahman, A.; Nandakumar, D.; Redino, C.; Choi, K.; Schiller, R.; Bhattacharya, S.; Veeramani, B.; et al. 2023. Anomaly detection via federated learning. In *2023 33rd International Telecommunication Networks and Applications Conference*, 259–266. IEEE.

Wang, J.; Liu, Q.; Liang, H.; Joshi, G.; and Poor, H. V. 2020. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *Advances in Neural Information Processing Systems*, volume 33.

Yamazaki, M.; Kora, Y.; Nakao, T.; Lei, X.; and Yokoo, K. 2022. Deep feature compression using rate-distortion optimization guided autoencoder. In *IEEE International Conference on Image Processing*, 1216–1220. IEEE.

Zhang, C.; Xie, Y.; Bai, H.; Yu, B.; Li, W.; and Gao, Y. 2021. A survey on federated learning. *Knowledge-Based Systems*, 216: 106775.

Zhou, Y.; Liang, X.; Zhang, W.; Zhang, L.; and Song, X. 2021. VAE-based deep SVDD for anomaly detection. *Neurocomputing*, 453: 131–140.

Zong, B.; Song, Q.; Min, M. R.; Cheng, W.; Lumezanu, C.; Cho, D.; and Chen, H. 2018. Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In *International conference on learning representations*.

Appendix Contents

A Implementation Details and Experimental Protocols

This appendix specifies the shared experimental settings and the method-specific implementations used in the reported evaluations. Data partitions, architectures, optimization settings, training budgets, and participation schedules are matched where applicable.

Common Experimental Settings

Architectures and optimization. For tabular datasets, we use a feed-forward autoencoder with encoder dimensions $d_x \to 256 \to 64 \to 16$ and a symmetric decoder, ReLU activations, and a linear output layer. For CIFAR-10 and Fashion-MNIST, we use a three-layer convolutional encoder with 128, 128, and 16 output channels and a symmetric decoder; the flattened latent dimensions are 1,024 and 784, and the models contain 339,219 and 334,609 parameters, respectively. All methods use Adam with learning rate 10^{-3}, mini-batch size 50, StepLR step size 1,000 and multiplier 0.1, and three random seeds $\{100, 200, 300\}$. For both vision datasets, seeds 100, 200, and 300 select normal/anomaly class pairs $2/8$, $0/4$, and $9/5$; CIFAR-10 uses 5,000 normal training samples (250 per client), while Fashion-MNIST uses 6,000 (300 per client). Thus, for the vision benchmarks, each seed jointly specifies the random state and the normal/anomaly class pair. The variation across the three runs therefore reflects both stochastic variation and variation across these three task instantiations rather than repeated training on one fixed class pair.

Training budget and participation. Training uses $R = 100$ global rounds. GCA performs $T_{\text{recon}} = 5$ reconstruction epochs followed by $T_{\text{align}} = 5$ alignment epochs per round. Single-Client, Centralized, FedAvg, FedProx, FedNova, and DP-FedAvg use 10 local epochs per round or the centralized epoch-equivalent. Unless otherwise stated, all decentralized methods use full client participation.

Data preprocessing and anomaly decisions. Training uses normal data only. Each client scores a sample using reconstruction error and sets its decision threshold to the 0.75 quantile of its training reconstruction errors. Tabular preprocessing removes features with Pearson correlation greater than 0.6 and applies standard normalization fitted on the normal training data.

Data Partitioning and Configuration Selection

Uniform-shard setting. For the primary IID experiments, the normal training data are randomly partitioned into disjoint uniform shards. The same realized partition is reused by every method for a given dataset and random seed.

Non-IID Dirichlet setting. For the heterogeneous experiments, client preferences are sampled from a Dirichlet distribution with concentration parameter $\alpha = 0.1$. Normal samples are represented by feature descriptors and grouped into $\min(20, C)$ groups using MiniBatchKMeans with 10 initializations, at most 200 iterations, and batch size $\min(2{,}048, N)$; group-specific client preferences are then allocated under long-tail quotas with a target maximum-to-minimum ratio of 10:1. Tabular descriptors are the standardized features, whereas vision descriptors are normalized pixels adaptively pooled to 8×8; only the five tabular non-IID results are reported. The same realized partition is reused by every method for a given dataset and seed. Because this experiment changes client data distributions rather than client feature dimensions, we refer to it as non-IID statistical heterogeneity.

Accuracy reporting and GCA component-count selection. GCA is evaluated after each reconstruction and alignment epoch, whereas the baselines are evaluated after each round. For each seed, we report the highest reconstruction-epoch test accuracy for GCA and the highest round-level test accuracy for each baseline. For GCA-GMM, $K^\star$ maximizes the three-seed mean over $K \in \{10, 20, 40, 80\}$, with exact ties resolved in favor of the smaller K. The same dataset-specific $K^\star$ is used for the main IID comparison and the extraction evaluation, while communication is reported for every tested K. The weighting diagnostic reports the highest alignment-epoch accuracy at this $K^\star$.

GCA Implementation

Local reconstruction and latent upload. Each participating client first updates both its encoder and decoder using reconstruction loss. It then uploads a uniformly sampled subset of its normal latent codes, with upload fraction ρ_i given in the main experimental setup. No raw data, model parameters, or gradients are uploaded.

Latent-coordinate compatibility. For each dataset and seed, clients use the same encoder architecture, latent dimension, and preprocessing. The experiments therefore assume that the resulting latent coordinates are sufficiently compatible for pooled clustering. Arbitrary client-specific permutations or transformations of the latent space are not considered.

Server clustering and broadcast. The server pools the uploaded latent codes and fits one of four clustering backends: GMM, K-means, Soft K-means, or Mini-Batch K-means. For every backend, $K \in \{10, 20, 40, 80\}$. The server broadcasts all K returned centroids and their support counts. For a GMM, $N_k = \sum_n \gamma_{nk}$ is the effective support count; for hard clustering, N_k is the cluster size.

For GMM, component means are initialized with Torch K-means++. Tabular datasets use a Torch full-covariance GMM with regularization 10^{-6} and one initialization, while vision datasets use a Torch diagonal-covariance GMM with regularization 0.1

and 100 initializations. EM runs for at most 200 iterations with tolerance 10^{-3}. K-means uses 10 initializations; Soft K-means uses temperature 1.0 after K-means initialization with 10 initializations; and Mini-Batch K-means uses batch size 4,096 and 10 initializations.

Inverse-support centroid alignment. Each client assigns its local codes to the nearest centroid and scales each per-sample alignment term using an inverse-support weight derived from the assigned centroid's effective count. Let

$$\widehat{N}_k = \max(N_k, \epsilon), \qquad \bar{N} = \frac{1}{K}\sum_{k=1}^{K} \widehat{N}_k,$$

$$\widetilde{w}_k = \min\left\{ w_{\max}, \left(\frac{\bar{N}}{\widehat{N}_k}\right)^{\gamma} \right\},$$

$$w_k = \frac{\widetilde{w}_k}{\frac{1}{K}\sum_{j=1}^{K} \widetilde{w}_j + \epsilon}, \qquad \gamma = 1.0, \quad w_{\max} = 6.0, \quad \epsilon = 10^{-6}.$$

The normalization gives $K^{-1}\sum_k w_k \approx 1$. Alignment updates only the encoder, while the decoder remains local and inference continues to use reconstruction-error thresholding.

Uniform-weight GMM ablation. To isolate the contribution of inverse-support weighting, we compare the default GCA-GMM method with a uniform-weight variant in which $w_k = 1$ for every nonempty centroid. Both variants use the same $K \in \{10, 20, 40, 80\}$ sweep, data partitions, latent uploads, GMM fitting, nearest-centroid assignments, training schedule, and optimization settings. This ablation is limited to the GMM backend.

Baseline Implementations

Single-Client. Each client trains an autoencoder only on its local normal data, with no server communication or aggregation. The architecture, optimizer, and total local epoch budget match the decentralized methods. Reported performance is averaged across clients.

Centralized. A single autoencoder is trained on the pooled normal training data using the same architecture and optimizer. Its training budget is matched to the epoch-equivalent used by the decentralized methods. Centralized training is used as a reference and is not a privacy-preserving deployment.

FedAvg. Clients train the complete autoencoder locally and upload model parameters or parameter deltas. The server aggregates client updates using local-sample-size weighting and broadcasts the resulting global autoencoder (McMahan et al. 2017).

FedProx. FedProx follows the FedAvg communication protocol and adds a proximal penalty to the complete autoencoder parameters (Li et al. 2020b). The local objective is

$$\mathcal{L}^{(i)}_{\text{FedProx}} = \mathcal{L}^{(i)}_{\text{recon}} + 10^{-4}\left\|\theta_i - \theta^{(r)}\right\|_2^2,$$

where the effective coefficient 10^{-4} results from the configured $\mu = 10^{-2}$ under the implemented loss scaling.

FedNova. FedNova uses the same local autoencoder, optimizer, participation schedule, and local epoch budget as FedAvg. Each client transmits a normalized local update, and the server applies normalized aggregation following FedNova (Wang et al. 2020). With 10 local epochs, $\tau_i = 10 n_i^{\text{batch}}$, and the effective normalization is $\tau_{\text{eff}} = \sum_i p_i \tau_i$, where p_i is the sample-size weight of client i.

DP-FedAvg. DP-FedAvg follows the FedAvg training and communication protocol while clipping each client's full model delta to global ℓ_2 norm 1.0 at the server. After sample-size-weighted aggregation, the server adds independent Gaussian noise to each aggregate tensor with standard deviation σ/C, where $\sigma = 1.0$ and C is the number of clients. Because the current implementation does not include a complete sampling and composition accountant, this configuration is treated as a noisy-update utility and extraction baseline rather than a certified (ε, δ)-DP mechanism (Abadi et al. 2016; McMahan et al. 2018).

Server-Side Extraction Attack Implementations

Parameter-sharing baselines. For FedAvg, FedProx, FedNova, and DP-FedAvg, the server has white-box access to one victim client's selected round-100 autoencoder and performs no additional local update. Using the same 100 attack initializations across all methods, the server minimizes reconstruction error with Adam at learning rate 10^{-2} for at most 1000 iterations. Each initialization uses independent best-state tracking and stops after 10 consecutive steps with relative improvement below 10^{-3}. Vision inputs are projected to $[-1, 1]$, whereas standardized tabular inputs are left unbounded.

Table 1: Implementation summary for all compared approaches.

Method	Client objective	Communicated object	Server operation	Key setting
Single-Client	Reconstruction	None	None	10 local epochs
Centralized	Reconstruction	Pooled training data	Central training	Epoch-equivalent budget
FedAvg	Reconstruction	AE model update	Sample-size-weighted averaging	10 local epochs
FedProx	Reconstruction plus proximal term	AE model update	Sample-size-weighted averaging	Configured $\mu = 10^{-2}$; effective coefficient 10^{-4}
FedNova	Reconstruction	Normalized AE update	Normalized aggregation	$\tau_i = 10n_i^{\mathrm{batch}}$; sample-size-weighted normalization
DP-FedAvg	Reconstruction	Clipped and noisy AE update	FedAvg aggregation	clip 1.0; $\sigma = 1.0$
GCA	Reconstruction plus centroid alignment	Latent codes / centroid statistics	Clustering	$K \in \{10, 20, 40, 80\}$
GCA-GMM (uniform weights)	Reconstruction plus centroid alignment	Latent codes / centroid statistics	GMM clustering	$w_k = 1$; $K \in \{10, 20, 40, 80\}$

GCA latent-only attack. Starting from the same client's round-100 checkpoint, GCA performs five reconstruction epochs and uploads the configured latent subset. The server trains an independently initialized surrogate autoencoder using only these latent codes, with Adam at learning rate 10^{-2}, batch size 50, a 10% validation split, at most 1000 epochs, minimum relative improvement 10^{-4}, and patience 20. It then applies the same 100-start input-space optimization, early-stopping rule, and input-domain constraints used for the parameter-sharing methods. The surrogate receives no victim parameters, gradients, or raw records.

Metrics. The reported attack output is $T_i(\mathbf{x}^\star)$ for the parameter-sharing methods and $\widetilde{T}_i(\mathbf{x}^\star)$ for GCA, rather than the optimized input $\mathbf{x}^\star$ itself. For every target in the evaluated client's training shard, we select its MSE-nearest output and report normalized target MSE and excess cosine similarity relative to 100 normal records excluded from training. Cosine similarity is computed using the same MSE-selected output. Means and standard deviations are reported over the same three seed configurations used in the performance evaluation.

Deterministic Communication Accounting

We compare the deterministic per-round tensor payload of GCA-GMM for every tested $K \in \{10, 20, 40, 80\}$ with FedAvg, FedProx, FedNova, and DP-FedAvg. Let C denote the number of clients, U the total uploaded latent codes, D_z the flattened latent dimension, and P the complete autoencoder parameter count. All communicated tensors are FP32. GCA uploads UD_z latent scalars and broadcasts $K(D_z + 1)$ centroid-and-count scalars to each client; each FL baseline uploads and broadcasts P model scalars per client. The totals include every client copy but exclude transport headers, serialization overhead, acknowledgements, checkpoint I/O, and optimizer state. This accounting reports per-round tensor payload only and does not measure runtime or total communication to a target accuracy.

Computing Environment

Experiments were executed on NVIDIA H100 and H200 GPUs. The same numerical configurations and software environment were used across accelerator types.

Experimental Controls

For a given dataset and seed configuration, every method uses the same training and test data and the same client partition. Autoencoder capacity, local optimizer, batch size, local epoch budget, and client participation are matched where applicable. GCA-GMM uses the same selected $K^\star$ in the main performance and extraction evaluations, while communication is reported for every tested K. All methods use the same three seed configurations; for the vision datasets, these configurations also specify the normal/anomaly class pairs described above.

B Positioning Relative to Prototype-Based FL and Differentially Private AE-FL

Class-Prototype Aggregation versus GCA

FedProto communicates class-conditioned feature prototypes (Tan et al. 2022). For each observed class, a client computes a local prototype as the mean of the corresponding sample embeddings. The server then aggregates prototypes having the same class label, using their local class support, and returns global class prototypes. Each client optimizes a supervised classification loss together with a regularization term that moves its local class prototypes toward the corresponding global ones.

GCA shares only the broad idea of communicating representation-space summaries. Unlike FedProto, it operates on unlabeled normal data, uploads sampled latent codes rather than class-indexed means, and discovers round-specific geometric modes by clustering. Clients align their encoder outputs with the resulting centroids using inverse-support weights; decoders remain local, and inference uses reconstruction error.

Table 2: Methodological distinction between the original supervised FedProto formulation (Tan et al. 2022) and GCA.

Dimension	FedProto-style class-prototype FL	GCA
Learning task	Supervised classification	One-class, unlabeled anomaly detection
Semantic identity	Class labels identify corresponding prototypes across clients	No labels; latent modes are discovered from the pooled codes
Client uplink	One local mean embedding for each observed class	Sampled individual latent codes from normal data
Server operation	Aggregate local prototypes that share the same class label	Fit a clustering model to pooled latent codes
Server downlink	Global class prototypes	Global latent centroids and effective support counts
Use of support	Local class counts weight the aggregation of same-label prototypes	Inverse global support scales local alignment through $w_k \propto 1/N_k$ with mean-one normalization
Client objective	Classification loss plus class-prototype regularization	AE reconstruction loss plus encoder-only centroid alignment
Model requirements	Shared class semantics and a compatible prototype dimension; local architectures may differ	A common latent dimension and sufficiently compatible latent coordinates; local decoders remain independent
Inference	Classifier output or distance to class prototypes	Reconstruction error with a training-error quantile threshold
Server-visible information	Class-level feature means; no model parameters or gradients in the standard FedProto protocol	Sampled latent codes in the uplink and centroid statistics in the downlink; no model parameters or gradients

The original FedProto formulation requires class identities and is therefore not directly applicable to GCA's unlabeled, one-class setting. GCA avoids parameter and gradient exchange, but its sampled latent codes remain information-bearing and are evaluated under the latent-only extraction threat model. Pooling latent codes further assumes a common latent dimension and sufficiently compatible latent coordinates across clients; centroid communication alone does not guarantee robustness to arbitrary feature-space heterogeneity.

Differential Privacy versus Withholding the AE Reconstruction Map

Differential privacy and communication architecture address different exposure mechanisms. DP-SGD clips individual contributions, adds calibrated noise, and accounts for privacy loss across optimization steps (Abadi et al. 2016). User-level DP-FL similarly clips and perturbs client contributions while composing privacy loss across communication rounds (McMahan et al. 2018). When the protected unit, adjacency relation, clipping rule, sampling process, noise mechanism, and privacy accountant are fully specified, these methods provide a formal bound on how much the distribution of the released model can change when one protected record or client is modified.

Such a guarantee does not, however, remove the trained AE's reconstruction map from the server-visible interface when the final encoder and decoder are communicated. The AE is still optimized so that $D(E(\mathbf{x})) \approx \mathbf{x}$ on normal training data, and a server possessing the model can directly evaluate $D \circ E$ or optimize candidate inputs toward low reconstruction error. A finite (ε, δ) guarantee does not assert that the trained model can never generate a training-like reconstruction. Rather, DP limits the influence of the protected record or client on the released model. It may consequently reduce record-specific memorization and attack success, but stronger noise can also degrade the representation and reduce anomaly-detection utility. White-box reconstruction therefore remains a relevant empirical attack surface for a DP-trained AE whose complete model is visible to the server (Ghoshal et al. 2025). A noise multiplier by itself is not a certified privacy guarantee. Computing an (ε, δ) budget additionally depends on the clipping granularity, sampling rate, number of rounds, client participation, adjacency definition, and accounting method (Abadi et al. 2016; McMahan et al. 2018). The complete DP-FedAvg implementation, including clipping, noise, and accounting status, is specified together with all other approaches in Appendix A.

GCA makes a complementary systems choice. It withholds AE parameters and gradients from the server, thereby preventing direct white-box access to the client reconstruction maps, but it exposes sampled latent codes instead. GCA therefore does not eliminate privacy risk and does not provide a formal DP guarantee. Its privacy properties are evaluated under a latent-specific

extraction threat model. In future work, a formal DP mechanism could be incorporated by clipping and perturbing the uploaded latent codes, privatizing the released centroid statistics, and accounting for their repeated release across communication rounds.

C Theoretical Analysis

We analyze one global round of GCA for a client i under the schedule of the *Methodology* section. We set the number of local epochs per phase to one, i.e., $T_{\text{recon}} = T_{\text{align}} = 1$. Thus, each round consists of exactly one full-batch gradient-descent step on the *reconstruction* loss (2) (encoder+decoder), full-latent code uploading (i.e., $\rho_i = 1$), and one full-batch gradient-descent step on the *alignment* loss (5) (encoder only). We establish a linear (geometric) decrease of each phase-specific objective (Bottou, Curtis, and Nocedal 2018), and then combine them to obtain a round-wise alternating gradient method convergence rate (Tseng 2001; Bolte, Sabach, and Teboulle 2014). We provide two analysis frameworks: first, a convex convergence analysis for which we show stable convergence towards a minimum; second, a non-convex convergence analysis for which we show convergence towards a critical point in finite time.

Convex Full-batch Convergence

Notation and Objectives. For client i, let $\theta_E \in \mathbb{R}^{p_E}$ and $\theta_D \in \mathbb{R}^{p_D}$ denote the parameters of the encoder $E_{\theta_E}(\cdot)$ and decoder $D_{\theta_D}(\cdot)$, respectively. The reconstruction objective is $F(\theta_E, \theta_D)$ as in (2), and the alignment objective $G(\theta_E; \mathcal{M}, \mathbf{w})$ as in (5).

Assumptions. We make smoothness/regularity assumptions to analyze alternating gradient methods around a local basin.
• **(A1)** L_F**-smoothness of** F**.** F has L_F-Lipschitz continuous gradients in a neighborhood $\mathcal{B}$ of the iterates:

$$\|\nabla F(\vartheta) - \nabla F(\vartheta')\| \le L_F \|\vartheta - \vartheta'\|$$

for all $\vartheta = (\theta_E, \theta_D), \vartheta' \in \mathcal{B}$.
• **(A2) Polyak-Łojasiewicz (PŁ) condition for** F**.** There exists $\mu_F > 0$ such that for all $(\theta_E, \theta_D) \in \mathcal{B}$:

$$\frac{1}{2}\|\nabla F(\theta_E, \theta_D)\|^2 \ge \mu_F\Big(F(\theta_E, \theta_D) - F^\star\Big), \qquad F^\star \triangleq \inf_{\mathcal{B}} F.$$

• **(A3) Fixed latent centroids and stable assignments (piecewise smoothness/PŁ).** Across all global rounds, treat $(\mathcal{M}, \mathbf{w})$ as fixed constants during the alignment epoch. Let the nearest-centroid assignment be $a(\mathbf{x})$ (cf.a_b in (3)). Assume there exists an open set (a fixed-assignment region)

$$\mathcal{R} \triangleq \big\{\theta_E : a(\mathbf{x}; \theta_E) = a(\mathbf{x}; \theta_E^+) \text{for all } x \in \mathcal{D}_i\big\}$$

containing the encoder iterates during the alignment step. On $\mathcal{R}$, the alignment objective $G(\cdot; \mathcal{M}, \mathbf{w})$ is L_G-smooth and satisfies a PŁ inequality with parameter $\mu_G > 0$:

$$\frac{1}{2}\big\|\nabla_{\theta_E} G(\theta_E; \mathcal{M}, \mathbf{w})\big\|^2 \ge \mu_G\big(G(\theta_E; \mathcal{M}, \mathbf{w}) - G^\star\big),$$

where $G^\star \triangleq \min_{\theta_E \in \mathcal{R}} G(\theta_E; \mathcal{M}, \mathbf{w})$ (Karimi, Nutini, and Schmidt 2016).
• **(A4) Bounded, mean-one weights.** The inverse-count weights are normalized as in (4), satisfy $\frac{1}{K}\sum_k w_k = 1$, and are bounded:

$$0 < w_{\min} \le w_k \le w_{\max} < \infty.$$

• **(A5) Fixed latent centroids and uniform latent proximity.** There exists a fixed latent-centroid set $\mathcal{M} = \{\mu_k\}_{k=1}^K$ such that

$$\sup_{\mathbf{x}} \min_k \big\|E_{\theta_E}(\mathbf{x}) - \mu_k\big\| \le \varepsilon_\mu.$$

That is, every encoding lies within an ε_μ-neighborhood of some centroid; if the minimum separation $\gamma \triangleq \min_{k \ne j} \|\mu_k - \mu_j\|$ satisfies $\varepsilon_\mu < \gamma/2$, nearest–centroid assignments are unique and stable (consistent with (A3)).

Phase-wise descent. For any L-smooth H and update $\theta^+ = \theta - \eta g(\theta)$, yields the standard linear (geometric) one-step decrease

$$H(\theta^+) - H^\star \le (1 - \eta^{(r)}\mu)\big(H(\theta) - H^\star\big), \qquad 0 < \eta^{(r)} \le \frac{1}{Lr}. \tag{15}$$

Lemma 1 (Reconstruction phase contraction)**.** *For the full-batch GD update* $\Theta^+ = \Theta - \eta^{(r)}_{recon}\nabla F(\Theta)$*, one step satisfies*

$$F(\Theta^+) - F^\star \le (1 - \eta^{(r)}_{recon}\mu_F)\big(F(\Theta) - F^\star\big),$$

where Θ *is the start-of-round parameter and* Θ^+ *is the parameter after the reconstruction step.*
Proof. *Apply* (15) *to* $H = F$*,* $L = L_F$*,* $\mu = \mu_F$*.* □

Lemma 2 (Alignment phase contraction with nearest assignments and mean-one weights)**.** *Fix the broadcast* $(\mathcal{M}, \mathbf{w})$ *within the round and assuming (A3)-(A5), the encoder-only update satisfies*

$$G(\theta_E^{++}; \mathcal{M}, \mathbf{w}) - G^\star \le (1 - \eta^{(r)}_{align}\mu_G)\big(G(\theta_E^+; \mathcal{M}, \mathbf{w}) - G^\star\big) + c_1\eta^{(r)}_{align}\varepsilon_\mu^2, \tag{16}$$

Algorithm 1: Global Centroid Alignment (GCA).

Require: Clients $\{\mathcal{D}_i\}_{i=1}^{C}$, AEs $\{(E_i, D_i)\}_{i=1}^{C}$; rounds R; local epochs $E = T_{\text{recon}} + T_{\text{align}}$; batch size B; upload fractions $\{\rho_i \in (0,1]\}$; GMM components K; learning rates $\eta_{\text{recon}}, \eta_{\text{align}}$; $\gamma = 1.0$; $w_{\max} = 6.0$; $\epsilon = 10^{-6}$

Ensure: Trained autoencoders; broadcast latent centroids and support counts

Initialize $\{\theta_{E_i}, \theta_{D_i}\}$ for all clients
for $r = 1$ to R **do**
  Server samples participating clients $\mathcal{S}_r$

  Phase 1: Reconstruction (client-side)
  for all clients $i \in \mathcal{S}_r$ **in parallel do**
    for $e = 1$ to T_{recon} **do**
      for all mini-batches $B_i = \{\mathbf{x}_b\}_{b=1}^{B} \subset \mathcal{D}_i$ **do**
        $\mathbf{z}_b \leftarrow E_i(\mathbf{x}_b), \quad \hat{x}_b \leftarrow D_i(\mathbf{z}_b)$
        $\mathcal{L}_{\text{recon}}^{(i)} \leftarrow \frac{1}{B}\sum_{b=1}^{B} \|\mathbf{x}_b - \hat{x}_b\|_2^2$
        $\theta_{E_i} \leftarrow \theta_{E_i} - \eta_{\text{recon}} \nabla_{\theta_{E_i}} \mathcal{L}_{\text{recon}}^{(i)}$
        $\theta_{D_i} \leftarrow \theta_{D_i} - \eta_{\text{recon}} \nabla_{\theta_{D_i}} \mathcal{L}_{\text{recon}}^{(i)}$
      end for
    end for
  end for
  for all clients $i \in \mathcal{S}_r$ **in parallel do**
    Select index set $\mathcal{I}_i$ with $|\mathcal{I}_i| = \lfloor \rho_i N_i \rfloor$
    $\mathbf{z}_i^{\uparrow} \leftarrow \{E_i(x) : x \in \mathcal{D}_i, \text{ indices } \mathcal{I}_i\}$
    Upload $\mathbf{z}_i^{\uparrow}$ to server
  end for

  Phase 2: Aggregation (server-side)
  $Z \leftarrow \bigcup_{i \in \mathcal{S}_r} \mathbf{z}_i^{\uparrow} = \{\mathbf{z}_n\}_{n=1}^{N_r}, \quad N_r \leftarrow \sum_{i \in \mathcal{S}_r} |\mathbf{z}_i^{\uparrow}|$
  Fit clustering algorithm with Z
  Compute counts N_k and centroids μ_k
  Broadcast $\{\mu_k, N_k\}_{k=1}^{K}$ to clients in $\mathcal{S}_r$

  Phase 3: Alignment (client-side, encoder-only)
  for all clients $i \in \mathcal{S}_r$ **in parallel do**
    $\widehat{N}_k \leftarrow \max(N_k, \epsilon)$, $\bar{N} \leftarrow \frac{1}{K}\sum_{k=1}^{K} \widehat{N}_k$
    $\tilde{w}_k \leftarrow \min\{w_{\max}, (\bar{N}/\widehat{N}_k)^{\gamma}\}$, $w_k \leftarrow \frac{\tilde{w}_k}{\frac{1}{K}\sum_{j=1}^{K} \tilde{w}_j + \epsilon}$
    for $e = 1$ to T_{align} **do**
      for all mini-batches $B_i = \{\mathbf{x}_b\}_{b=1}^{B}$ **do**
        $\mathbf{z}_b \leftarrow E_i(\mathbf{x}_b)$
        $a_b \leftarrow \arg\min_{k \in \{1,\dots,K\}} \|\mathbf{z}_b - \mu_k\|_2$
        $\mathcal{L}_{\text{align}}^{(i)} \leftarrow \frac{1}{B}\sum_{b=1}^{B} w_{a_b} \|\mathbf{z}_b - \mu_{a_b}\|_2^2$
        $\theta_{E_i} \leftarrow \theta_{E_i} - \eta_{\text{align}} \nabla_{\theta_{E_i}} \mathcal{L}_{\text{align}}^{(i)}$
      end for
    end for
  end for
end for

where c_1 is only bounded by (A3)–(A4); one may use $c_1 = \mu_G w_{\max}$.

Proof. *On the fixed-assignment region from (A3), $G(\cdot; \mathcal{M}, \mathbf{w})$ is L_G-smooth and μ_G-PŁ, hence the exact gradient step yields*

$$G(\theta_E^{++}) - G^\star \le (1 - \eta_{align}^{(r)} \mu_G)\big(G(\theta_E^{+}) - G^\star\big).$$

By (A5), every encoding lies within ε_μ of some centroid, hence

$$G(\theta_E; \mathcal{M}, \mathbf{w}) = \frac{1}{|\mathcal{D}_i|}\sum_{x} w_{a(x)} \|E_{\theta_E}(x) - \mu_{a(x)}\|^2 \le w_{\max} \varepsilon_\mu^2$$

for all iterates in the region; in particular, $G^\star \le w_{\max}\varepsilon_\mu^2$. Adding the nonnegative slack $\eta_{align}^{(r)}\mu_G G^\star$ to the right-hand side and using the bound on $G^\star$ gives (16). □

Across-Round Convergence of the Two-Phase Schedule. We now show that iterating GCA rounds converges, even though each round alternates between two different objectives.

Along with (A1)-(A5), assume the local curvature conditions in the basin of interest:

• **(A6)** F is μ_F-strongly convex and L_F-smooth on the basin containing the iterates.

• **(A7)** With fixed latent assignments, $G(\cdot;\mathcal{M},\mathbf{w})$ is μ_G-strongly convex and L_G-smooth in θ_E. Under (A6), for any such $H \in \{F, G\}$ we have the gradient-suboptimality sandwich

$$2\mu_H\big(H(\theta) - H^\star\big) \le \|\nabla H(\theta)\|^2 \le 2L_H\big(H(\theta) - H^\star\big). \tag{17}$$

Let $\Theta \triangleq (\theta_E, \theta_D)$ and define suboptimalities $\Delta_F(\Theta) \triangleq F(\Theta) - F^\star$ and $\Delta_G(\theta_E) \triangleq G(\theta_E;\mathcal{M},\mathbf{w}) - G^\star$. Within one round, recall:

$$\Theta \xrightarrow[\text{recon}]{\eta_{\text{recon}}^{(r)}} \Theta^+ = (\theta_E^+, \theta_D^+) \quad \text{and} \quad \Theta^+ \xrightarrow[\text{align}]{\eta_{\text{align}}^{(r)}} \Theta^{++} = (\theta_E^{++}, \theta_D^+).$$

Lemma 3 (Effect of a reconstruction step on G)**.** *Run one full-batch GD step on F. Then, for any $\zeta > 0$,*

$$\Delta_G(\theta_E^+) \le \Delta_G(\theta_E) + \eta_{recon}^{(r)}\frac{L_G}{\zeta}\Delta_G(\theta_E) + \eta_{recon}^{(r)}\,(\zeta L_F)\,\Delta_F(\Theta) + O((\eta_{recon}^{(r)})^2). \tag{18}$$

where $\Delta_G(\theta_E) \triangleq G(\theta_E;\mathcal{M},\mathbf{w}) - G^\star$.

Proof. *By L_G-smoothness on the encoder coordinate,*

$$G(\theta_E^+) \le G(\theta_E) + \big\langle \nabla_{\theta_E} G(\theta_E), \theta_E^+ - \theta_E\big\rangle + \frac{L_G}{2}\|\theta_E^+ - \theta_E\|^2.$$

Since $\theta_E^+ - \theta_E = -\eta_{recon}^{(r)}\nabla_{\theta_E} F(\Theta)$, apply Young's inequality to the cross term and use (17) *and $\|\nabla_{\theta_E} F\| \le \|\nabla F\|$ to obtain* (18). □

Theorem 1 (Effect of an alignment step on F)**.** *Run one encoder-only alignment step*

$$\theta_E^{++} = \theta_E^+ - \eta_{align}^{(r)}\nabla_{\theta_E} G(\theta_E^+;\mathcal{M},\mathbf{w}).$$

Then, for any $\zeta' > 0$,

$$\Delta_F(\Theta^{++}) \le \Delta_F(\Theta^+) + \eta_{align}^{(r)}\frac{L_F}{\zeta'}\Delta_F(\Theta^+) + \eta_{align}^{(r)}\big(\zeta' L_G\big)\Delta_G(\theta_E^+) + O((\eta_{align}^{(r)})^2), \tag{19}$$

where $\Delta_F(\Theta) \triangleq F(\Theta) - F^\star$.

Proof. *By L_F-smoothness of F in the encoder coordinate,*

$$F(\Theta^{++}) \le F(\Theta^+) + \big\langle \nabla_{\theta_E} F(\Theta^+), \theta_E^{++} - \theta_E^+\big\rangle + \frac{L_F}{2}\|\theta_E^{++} - \theta_E^+\|^2.$$

Substitute $\theta_E^{++} - \theta_E^+ = -\eta_{align}^{(r)}\nabla_{\theta_E} G(\theta_E^+)$ and apply Young's inequality to get

$$F(\Theta^{++}) - F^\star \le \Delta_F(\Theta^+) + \eta_{align}^{(r)}\frac{1}{2\zeta'}\|\nabla_{\theta_E} F(\Theta^+)\|^2 + \eta_{align}^{(r)}\frac{\zeta'}{2}\|\nabla_{\theta_E} G(\theta_E^+;\mathcal{M},\mathbf{w})\|^2 + \frac{L_F}{2}(\eta_{align}^{(r)})^2\|\nabla_{\theta_E} G(\cdot)\|^2.$$

Using the gradient-suboptimality bounds (from strong convexity/smoothness in (A6)–(A7)),

$$\|\nabla_{\theta_E} F(\Theta^+)\|^2 \le 2L_F\Delta_F(\Theta^+) \quad \textit{and} \quad \|\nabla_{\theta_E} G(\theta_E^+)\|^2 \le 2L_G\Delta_G(\theta_E^+),$$

yields (19). □

For convenience, we write $\alpha_r \triangleq \eta_{\text{recon}}^{(r)}\frac{L_G}{\zeta}$, $\beta_r \triangleq \eta_{\text{recon}}^{(r)}\zeta L_F$, $\alpha_a \triangleq \eta_{\text{align}}^{(r)}\frac{L_F}{\zeta'}$, and $\beta_a \triangleq \eta_{\text{align}}^{(r)}\zeta' L_G$. Then,

$$\Delta_F(\Theta^{++}) \le \Big[1 - \mu_F\eta_{\text{recon}}^{(r)} + \alpha_a + \beta_a\beta_r - \alpha_a\mu_F\eta_{\text{recon}}^{(r)}\Big]\Delta_F(\Theta) + \beta_a(1+\alpha_r)\Delta_G(\theta_E),$$

$$\Delta_G(\theta_E^{++}) \le \Big[1 - \mu_G\eta_{\text{align}}^{(r)} + \alpha_r - \mu_G\eta_{\text{align}}^{(r)}\alpha_r\Big]\Delta_G(\theta_E) + \Big[\beta_r - \mu_G\eta_{\text{align}}^{(r)}\beta_r\Big]\Delta_F(\Theta) + c_1\eta_{\text{align}}^{(r)}\varepsilon_\mu^2.$$

Since $\alpha_r, \beta_r, \alpha_a, \beta_a = O(r^{-1})$, products like $\alpha_a\mu_F\eta_{\text{recon}}^{(r)}$, $\beta_a\beta_r$, $\mu_G\eta_{\text{align}}^{(r)}\alpha_r$, and $\mu_G\eta_{\text{align}}^{(r)}\beta_r$ are $O(r^{-2})$ and can be absorbed. Summing the two displays gives, with $S^{(r)} \triangleq \Delta_F^{(r)} + \Delta_G^{(r)}$,

$$S^{(r+1)} \le \Big[1 - \mu_F\eta_{\text{recon}}^{(r)} + \alpha_a + \beta_r\Big]\Delta_F^{(r)} + \Big[1 - \mu_G\eta_{\text{align}}^{(r)} + \alpha_r + \beta_a\Big]\Delta_G^{(r)} + c_1\eta_{\text{align}}^{(r)}\varepsilon_\mu^2 + O(r^{-2}). \tag{20}$$

We pick $\zeta = \sqrt{L_G/L_F}$ and $\zeta' = \sqrt{L_F/L_G}$, which minimize the cross coefficients. Then

$$\alpha_r + \beta_r = 2\sqrt{L_F L_G}\,\eta^{(r)}_{\text{recon}}, \quad \text{and} \quad \alpha_a + \beta_a = 2\sqrt{L_F L_G}\,\eta^{(r)}_{\text{align}}.$$

Using $\alpha_r, \beta_r \le \alpha_r + \beta_r$ and $\alpha_a, \beta_a \le \alpha_a + \beta_a$ in (20) and dropping the negative terms $-\mu_F \eta^{(r)}_{\text{recon}} \Delta^{(r)}_F$ and $-\mu_G \eta^{(r)}_{\text{align}} \Delta^{(r)}_G$ yields

$$S^{(r+1)} \le \Big(1 + \gamma_r\Big) S^{(r)} + c_1\, \eta^{(r)}_{\text{align}}\, \varepsilon^2_\mu + O(r^{-2}),$$

where $\gamma_r \triangleq 2\sqrt{L_F L_G}\big(\eta^{(r)}_{\text{recon}} + \eta^{(r)}_{\text{align}}\big)$.

We assume bounded objective gaps.

• **(A8)** There exist $S_{\max} < \infty$ and $r_0 \in \mathbb{N}$ such that

$$\sup_{r \ge r_0} S^{(r)} \le S_{\max}, \quad \text{where} \quad S^{(r)} \triangleq \Delta^{(r)}_F + \Delta^{(r)}_G.$$

Under (A8), from the one–step bound above we have, for all $r \ge r_0$,

$$S^{(r+1)} - S^{(r)} \le \gamma_r\, S_{\max} + c_1\, \eta^{(r)}_{\text{align}}\, \varepsilon^2_\mu + O(r^{-2}).$$

Using the admissible decays $\eta^{(r)}_{\text{recon}}$ and $\eta^{(r)}_{\text{align}}$ gives

$$S^{(r+1)} - S^{(r)} \le \frac{2\Big(\sqrt{\frac{L_G}{L_F}} + \sqrt{\frac{L_F}{L_G}}\Big)\, S_{\max}}{r} + \frac{\frac{c_1}{L_G}\, \varepsilon^2_\mu}{r} + O(r^{-2}),$$

We can also provide a lower bound for $S^{(r+1)}$ in a similar way.

Therefore, $\lim_{r\to\infty}\big(S^{(r+1)} - S^{(r)}\big) = 0$: the sequence $\{S^{(r)}\}$ is *stable* in the sense that, by (A8), it remains uniformly bounded.

Non-convex Full-batch Convergence

To show convergence to a stationary point in the error surface, we adapt the analysis of the Alternating Forward-Backward (AFB) algorithm (Frankel, Garrigos, and Peypouquet 2014), also known as PALM (Bolte, Sabach, and Teboulle 2014), as a benchmark to evaluate GCA. AFB is a proximal gradient descent method and treats the optimization problem

$$\underset{x_i \in X_i}{\text{minimize}}\, h(x_1, \ldots, x_p) = \sum_{i=1}^{p} f_i(x_i) + g(x_1, \ldots, x_p) \tag{21}$$

where the independent variables are split into blocks x_i and alternates in locally optimizing sub-objectives over blocks of variables and updating them, i.e.

$$x_i^{(r+1)} \in \text{prox}^{f_i}_{\alpha_{i,r}}\left(x_i^{(r)} - \frac{1}{\alpha_{i,r}} \nabla_i g(x_1^{(r+1)}, \ldots, x_{i-1}^{(r+1)}, x_i^{(r)}, \ldots, x_p^{(r)})\right) \tag{22}$$

where the proximal map consists of

$$\text{prox}^f_\alpha(\mathbf{x}) \triangleq \underset{\mathbf{y}}{\text{argmin}} \left\{ f(\mathbf{y}) + \frac{\alpha}{2} ||\mathbf{y} - \mathbf{x}||^2 \right\} \tag{23}$$

with inverse learning rate α. The sequence generated by AFB is known to be of finite length if the objective function h satisfies the Kurdyka-Łojasiewicz (KŁ) condition (defined below) everywhere in the domain of ∂h (known as a KŁ function), thus converging towards a critical point in h. The only other assumptions necessary for AFB are that g is L_g-smooth and f_i are proper lower semicontinuous functions, where g, f_i *do not have to be convex in any way*.

Notation and Objectives. For client i, let $\theta_{E_i} \in \mathbb{R}^{p_E}$ and $\theta_{D_i} \in \mathbb{R}^{p_D}$ denote the parameters of the encoder $E_i(\cdot) \triangleq E_i(\,\cdot\,; \theta_{E_i})$ and decoder $D_i(\cdot) \triangleq D(\,\cdot\,; \theta_{D_i})$, respectively. For simplicity, we concatenate the model parameters at each client i into a single $\theta \triangleq \{\theta_{E_i}, \theta_{D_i}\}$ and the clustering parameters into a single $\mu \triangleq \mathcal{M}$. We rename the reconstruction objective as in (2) using $F(\theta) = \sum_i \mathcal{L}^{(i)}_{\text{recon}}(\theta_{E_i}, \theta_{D_i}; \mathcal{D}_i)$, and we define a co-objective $G(\theta, \mu)$ which encompasses two requirements:

• When the encoder parameters are fixed, $G(\mu; \theta_E)$ should serve as a clustering objective measuring proximity between μ, a set of K points, and the "true" centroids of a GMM fitted onto the underlying data distribution projected onto the latent space through the encoder with parameters θ_E.

• When the centroid locations are fixed, $G(\theta_E;\mu)$, the alignment objective for each client i, $\mathcal{L}^{(i)}_{\text{align}}(\theta_{E_i};\mu,\mathcal{D}_i)$ as per (5), such that the partial gradient of G on the encoder parameters

$$\nabla_{\theta_E} G(\theta,\mu) \triangleq \left\{\frac{\partial G(\theta,\mu)}{\partial \theta_{E_i}}\right\} \triangleq \left\{\nabla \mathcal{L}^{(i)}_{\text{align}}(\theta_{E_i};\mu,\mathcal{D}_i)\right\}$$

is equal by definition to the set of gradients of the alignment loss on each client. Moreover, we assert that since the alignment step does not modify the decoder parameters, the partial gradient of the co-objective G on the decoder parameters is zero and we define the partial gradient of G on the model parameters in general as:

$$\begin{aligned}\nabla_{\theta_D} G(\theta,\mu) &\triangleq \left\{\frac{\partial G(\theta,\mu)}{\partial \theta_{D_i}}\right\} = 0\\ \nabla_{\theta} G(\theta,\mu) &\triangleq \Big(\nabla_{\theta_E} G(\theta,\mu), \nabla_{\theta_D} G(\theta,\mu)\Big)\\ &= \Big(\left\{\nabla \mathcal{L}^{(i)}_{\text{align}}(\theta_{E_i};\mu,\mathcal{D}_i)\right\}, 0\Big)\end{aligned}$$

Lemma 4 (Effective co-objective)**.** *The requirements for the co-objective $G(\theta,\mu)$ can be satisfied with the alignment loss of GCA alone:*

$$\mathcal{L}^{(i)}_{\textit{align}}(\mathcal{M},\mathbf{w};\theta_{E_i},\mathcal{D}_i) = \frac{1}{|\mathcal{D}_i|}\sum_{d=1}^{|\mathcal{D}_i|} w_{a_{i,d}} \left\|E_i(\mathbf{x}_{i,d}) - \mu_{a_{i,d}}\right\|_2^2$$

where $a_{i,d}$ indexes the latent centroid assigned to data point d at client i is assigned.

Proof. Recalling (3), (4), and (5), note that since computing GMM using the EM approximately solves

$$\min_{\mathcal{M},\mathbf{w},a_b} \mathcal{L}^{(i)}_{\text{align}}(\mathcal{M},\mathbf{w};\theta_{E_i},\mathcal{D}_i) = \min_{\mathcal{M},\mathbf{w},a_b} \frac{1}{|\mathcal{D}_i|}\sum_{d=1}^{|\mathcal{D}_i|} w_{a_{i,d}} \left\|E_i(\mathbf{x}_{i,d}) - \mu_{a_{i,d}}\right\|_2^2$$

and that $\nabla_\mu \mathcal{L}^{(i)}_{\text{align}}(\theta_E,\mu;\mathcal{D}_i)$ is piecewise L_H-continuous, the properties of objective $G_{\text{cl}}(\mu;\theta_E)$ can be fulfilled using $\mathcal{L}_{\text{align}}$. Since the alignment loss satisfies both the clustering objective and the alignment objective (by definition above), the alignment loss of GCA on its own can be used as the co-objective. □ □

Therefore, we define the co-objective $G(\theta,\mu)$ as

$$\begin{aligned}G(\theta,\mu) &\equiv \sum_i \mathcal{L}^{(i)}_{\text{align}}(\theta_{E_i},\mu;\mathcal{D}_i)\\ &= \sum_i \frac{1}{|\mathcal{D}_i|}\sum_{d=1}^{|\mathcal{D}_i|} w_{a_{i,d}} \left\|E_i(\mathbf{x}_{i,d}) - \mu_{a_{i,d}}\right\|_2^2, \qquad a_{i,d} \in \operatorname*{argmin}_k \|E_i(\mathbf{x}_{i,d}) - \mu_k\|_2,\end{aligned} \tag{24}$$

$$N_k = \sum_i \sum_{d=1}^{|\mathcal{D}_i|} \mathbf{1}(a_{i,d}=k), \quad \tilde{w}_k = \frac{\frac{1}{K}\sum_{j=1}^K N_j}{N_k}, \quad w_k = \frac{\tilde{w}_k}{\frac{1}{K}\sum_{j=1}^K \tilde{w}_j}.$$

We define the global objective function as $H(\theta,\mu) = F(\theta) + G(\theta,\mu)$.

Assumptions. As with AFB, we make few assumptions on the objective function.

• **(A1+) Properties of objective function.** G is L_G-smooth and F is L_F-smooth and a proper lower semicontinuous function, and note that F, G do not have to be convex in any way.

• **(A2+) Kurdyka-Łojasiewicz (KŁ) condition for G.** The KŁ property for a function g is satisfied at a given $x \in \operatorname{dom}(g)$ if there exist a neighborhood X around x, an $\eta \in (0,+\infty]$, and a concave and continuous function φ which satisfy $\varphi(0)=0$, continuous at 0, C^1 on $(0,\eta)$, and $\varphi' > 0\ \forall s \in (0,\eta)$, such that $\forall u \in U \cap [g(x) < g(u) < g(x)+\eta]$, the following condition holds:

$$\varphi'(g(u) - g(x))\operatorname{dist}(0,\partial g(u)) \geq 1 \tag{25}$$

We assume that G is a KŁ function, that is it satisfies the KŁ condition over all points in the domain of ∂G.

Phase-wise descent. We then rearrange the phases in GCA (see Algorithm 1) and define an AFB-like method with variable blocks (θ, μ) in where we repeat the following steps until convergence:

1. $\mu^{(r+1)}$ takes the centroids computed from GMM based on latent codes computed using $E(\theta^{(r)})$ (Phase 2 in GCA)
2. $\theta^{(r+1)}$ takes two gradient steps, one descending on $G(\theta, \mu)$, then descending on $F(\theta)$;
   - $\theta^{(r)+} = \theta^{(r)} - \eta^{(r)}_{\text{align}} \nabla_\theta G(\theta^{(r)}; \mu)$ as per Phase 3 in GCA, then
   - $\theta^{(r+1)} = \theta^{(r)+} - \eta^{(r)}_{\text{recon}} \nabla F(\theta^{(r)+})$ as per Phase 1 in GCA.

     We define $g^{(r)}(\theta^{(r)}) = \eta^{(r)}_{\text{align}} \nabla_\theta G(\theta^{(r)}; \mu) + \eta^{(r)}_{\text{recon}} \nabla F(\theta^{(r)+})$ as the total gradient step taken. Note that Phase 1 from the subsequent step in as GCA is implemented is shifted to the current step under this arrangement.

We note differences between GCA and AFB. First, in the update of μ, since we use the EM algorithm to update GMM centroids, we approximate the minimizer of $G(\mu; \theta)$ along μ with fixed θ.

Though it may result in a lower $H(\theta^{(r)}, \mu^{(r+1)}_{\text{GCA}}) \leq H(\theta^{(r)}, \mu^{(r+1)}_{\text{AFB}})$ (where $\mu^{(r+1)}_{\text{GCA}}$ is the centroid location update computed through EM in the GCA algorithm formulation, and $\mu^{(r+1)}_{\text{AFB}}$ is a theoretical update of μ computed as per the AFB algorithm) , it does not guarantee a better $H(\theta^{(r+1)}_{\text{GCA}}, \mu^{(r+1)}_{\text{GCA}})$ than $H(\theta^{(r+1)}_{\text{AFB}}, \mu^{(r+1)}_{\text{AFB}})$. As such, we define the difference between the GCA update for μ and the AFB update (just gradient descent since there is no proximal gradient) as

$$\delta^{(r)} = \mu^{(r+1)}_{\text{GCA}} - \mu^{(r)} + {\alpha^{(r)}_\mu}^{-1} \nabla_\mu G(\mu; \theta) \tag{26}$$

where ${\alpha^{(r)}_\mu}^{-1}$ is an arbitrary descent rate.

For the model parameter update, we recall the AFB update (C)

$$\theta^{(r+1)}_{\text{AFB}} \in \text{prox}^F_{\alpha^{(r)}_\theta} \left(\theta^{(r)} - {\alpha^{(r)}_\theta}^{-1} \nabla_\theta G(\theta^{(r)}; \mu) \right)$$

where ${\alpha^{(r)}_\theta}^{-1}$ is another arbitrary descent rate. This update rule is different from the GCA update due to the replacement of the proximal map for $F(\theta)$ with a subsequent gradient descent step. We therefore define the difference between the GCA update for θ and the AFB update as

$$\varepsilon^{(r+1)} = \theta^{(r+1)}_{\text{GCA}} - \theta^{(r+1)}_{\text{AFB}} \tag{27}$$

Proposition 1 (GCA as an imperfect form of AFB)**.** *Given the sequences $\delta^{(r)}$ and $\varepsilon^{(r)}$, we can model GCA as an AFB implementation with errors as per Section 4.2 of (Frankel, Garrigos, and Peypouquet 2014) and that GCA converges under typical training regimes with decaying learning rate.*

Proof. For the series $||\theta^{(r+1)}_{\text{GCA}} - \theta^{(r)}_{\text{GCA}}||$ to converge, the above work states that the following condition (**HE**) must be met: there exists a $\sigma \in [0, +\infty), \rho \in (0, 1]$ with $\frac{\sigma+1}{\rho} < \underline{\alpha}/L_G$ where $\underline{\alpha} > 0, \min\{\alpha^{(r)}_\theta, \alpha^{(r)}_\mu\} \geq \underline{\alpha} > L_G$ such that:

1. $\left\|\varepsilon^{(r)}\right\| \leq \frac{\sigma}{2} \left\|\theta^{(r+1)}_{\text{AFB}} - \theta^{(r)}_{\text{AFB}}\right\|$ where $\theta^{(r)}_{\text{AFB}}$ are defined in relation to the value from the previous step as computed by GCA.
2. $\left\|\delta^{(r)}\right\| \leq \frac{\sigma}{2} \left\|\mu^{(r+1)}_{\text{AFB}} - \mu^{(r)}_{\text{AFB}}\right\|$ where $\mu^{(r)}_{\text{AFB}} = \mu^{(r)} - {\alpha^{(r)}_\mu}^{-1} \nabla_\mu G(\mu; \theta)$ as per (C) and defined in relation to the value from the previous step as computed by GCA.
3. $\left\langle \varepsilon^{(r)}, \theta^{(r+1)}_{\text{AFB}} - \theta^{(r)}_{\text{AFB}} \right\rangle \leq \frac{1-\rho}{2} \left\|\theta^{(r+1)}_{\text{AFB}} - \theta^{(r)}_{\text{AFB}}\right\|^2$ and $\left\langle \delta^{(r)}, \mu^{(r+1)}_{\text{AFB}} - \mu^{(r)}_{\text{AFB}} \right\rangle \leq \frac{1-\rho}{2} \left\|\mu^{(r+1)}_{\text{AFB}} - \mu^{(r)}_{\text{AFB}}\right\|^2$.

We remark that the above conditions can be met if we select an increasing sequence $\alpha^{(r)}_\theta > L_G$ and superior sequences $1/\eta^{(r)}_{\text{recon}} > 2\alpha^{(r)}_\theta \ \forall r$ and $1/\eta^{(r)}_{\text{align}} > 2\alpha^{(r)}_\theta \ \forall r$, the conditions on $\varepsilon^{(r)}$ can be met.

For the centroid step, recall the co-objective $G(\theta, \mu)$ based on the alignment loss in (24) which is entirely composed of squared L_2 norms when θ is fixed. We thus know that its gradient and smoothness in μ are analytically obtainable and that a single-step gradient descent size $\alpha^{(r)}_\mu$ can be computed at each round; as such, we know that there exists a sequence $\alpha^{(r)}_\mu$ for which $\left\|\delta^{(r)}\right\|$ is negligible.

□

We thus know that GCA can be analyzed as an AFB-type algorithm with tractable errors, and following results in (Frankel, Garrigos, and Peypouquet 2014), we know that it can converge towards a critical point of $H(\theta, \mu)$. Note that this method shows a convergence towards the co-objective $H(\theta, \mu) = F(\theta) + G(\theta, \mu)$, not solely the reconstruction objective $F(\theta)$.

D Additional Theory for Server-Side Data Extraction Attacks

This appendix provides supporting theoretical results for the two server-side extraction routes introduced in the *Adversarial Attack Scenarios* section: (i) DeepDream-style inversion against a *white-box* victim autoencoder (standard FL), and (ii) DeepDream-style inversion against a surrogate trained *from scratch* on uploaded latent codes only (GCA).

Why reconstruction objectives admit training-like fixed points (standard FL)

Let $E : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ and $D : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$ be a trained autoencoder, and define the reconstruction map

$$T(\mathbf{x}) \triangleq D(E(\mathbf{x})). \tag{28}$$

In standard FL, the server-side DeepDream-style inversion in (11) minimizes the reconstruction energy

$$r(\mathbf{x}) \triangleq \|\mathbf{x} - T(\mathbf{x})\|_2^2 \text{ over } \mathbf{x} \in \mathcal{X}. \tag{29}$$

We analyze the geometry of $r(\mathbf{x})$ to explain why its minimizers and near-minimizers lie near the autoencoder's reconstruction set, which may reflect the training distribution when the autoencoder reconstructs its training data accurately.

Define the reconstruction residual

$$e(\mathbf{x}) \triangleq \mathbf{x} - T(\mathbf{x}) \in \mathbb{R}^{d_x}, \text{ so that } r(\mathbf{x}) = \|e(\mathbf{x})\|_2^2. \tag{30}$$

Assume T is differentiable on $\mathcal{X}$ and denote its Jacobian by $J_T(\mathbf{x}) \in \mathbb{R}^{d_x \times d_x}$. Then e is differentiable with $J_e(\mathbf{x}) = I - J_T(\mathbf{x})$, and by the chain rule,

$$\begin{aligned} \nabla r(\mathbf{x}) = 2\, J_e(\mathbf{x})^\top e(\mathbf{x}) &= 2\,(I - J_T(\mathbf{x}))^\top \big(\mathbf{x} - T(\mathbf{x})\big) \\ &= 2\,(I - J_T(\mathbf{x}))^\top e(\mathbf{x}). \end{aligned} \tag{31}$$

Fixed points and global minimizers. Define the set of fixed points (exact reconstructions)

$$\mathcal{F} \triangleq \{\mathbf{x} \in \mathcal{X} : T(\mathbf{x}) = \mathbf{x}\}. \tag{32}$$

By definition, $r(\mathbf{x}) = \|\mathbf{x} - T(\mathbf{x})\|_2^2 \geq 0$ and

$$r(\mathbf{x}) = 0 \iff T(\mathbf{x}) = \mathbf{x} \iff \mathbf{x} \in \mathcal{F}. \tag{33}$$

Consequently, if $\mathcal{F} \neq \emptyset$ (i.e., r attains value 0 on $\mathcal{X}$), then

$$\min_{\mathbf{x} \in \mathcal{X}} r(\mathbf{x}) = 0, \qquad \arg\min_{\mathbf{x} \in \mathcal{X}} r(\mathbf{x}) = \mathcal{F}. \tag{34}$$

If $\mathcal{F} = \emptyset$, then $r(\mathbf{x}) > 0$ for every $\mathbf{x} \in \mathcal{X}$. If the minimum is attained, its value is strictly positive, and its minimizers are the points with the smallest reconstruction residual.

Why fixed points can be training-like. If the autoencoder reconstructs its training records well, then for many training samples $\mathbf{x}_n$ we have $r(\mathbf{x}_n) \approx 0$. In such regimes, the reconstruction map T is close to the identity on the data manifold, and training samples (or nearby points on the learned manifold) behave as approximate fixed points of T. Therefore, directly minimizing $r(\mathbf{x})$ via gradient-based input optimization admits solutions that lie on (or near) the learned reconstruction set, which in turn is shaped by the training distribution.

Why latent-only flipping can still yield training-like reconstructions (GCA), and why it is harder than white-box inversion

This subsection formalizes why the latent-only "decoder–encoder flipping" route in the *Adversarial Attack Scenarios* section is fundamentally non-identifying: from uploaded latents alone, the server cannot uniquely determine the underlying private training records in input space. Consequently, unlike standard-FL white-box inversion, latent-only flipping cannot guarantee faithful reconstruction of client data without additional assumptions.

Let the (unknown) client autoencoder be (E, D) and let the server observe only a finite set of uploaded latent codes

$$Z \triangleq \{\mathbf{z}_n\}_{n=1}^M \subset \mathbb{R}^{d_z}, \qquad \mathbf{z}_n = E(\mathbf{x}_n), \tag{35}$$

for unknown private records $\{\mathbf{x}_n\}_{n=1}^M \subset \mathcal{X}$. A latent-only attacker is any (possibly randomized) mapping $\mathcal{A}$ that takes Z as input and outputs reconstructions $\hat{\mathbf{x}}_n = \mathcal{A}_n(Z) \in \mathcal{X}$.

What the flipping attacker optimizes. The flipping attack chooses a parametric surrogate pair $(\tilde{D}, \tilde{E})$ and trains it *only on* Z by minimizing a latent reconstruction objective of the form

$$\min_{\theta_{\tilde{D}}, \theta_{\tilde{E}}} \quad \frac{1}{M} \sum_{n=1}^{M} \left\| \mathbf{z}_n - \tilde{E}(\tilde{D}(\mathbf{z}_n)) \right\|_2^2. \tag{36}$$

After training, it defines $\widetilde{T} = \widetilde{D} \circ \widetilde{E}$ and applies the same input-space reconstruction objective used by standard FL. Importantly, (36) constrains only the composition $\widetilde{E} \circ \widetilde{D}$ on the finite observed set Z and never observes the corresponding $\mathbf{x}_n$.

Proposition 2 (Latent-only indistinguishability up to isometries)**.** *Fix any bijection $\phi : \mathcal{X} \to \mathcal{X}$ that preserves the ℓ_2 norm, i.e., $\|\phi(\mathbf{u}) - \phi(\mathbf{v})\|_2 = \|\mathbf{u} - \mathbf{v}\|_2$ for all $\mathbf{u}, \mathbf{v} \in \mathcal{X}$. Define transformed private records $\mathbf{x}'_n = \phi(\mathbf{x}_n)$ and a transformed encoder*

$$E'(\mathbf{x}) \triangleq E(\phi^{-1}(\mathbf{x})). \tag{37}$$

Then the induced latent set is identical*:*

$$\begin{aligned} E'(\mathbf{x}'_n) &= E(\phi^{-1}(\phi(\mathbf{x}_n))) \\ &= E(\mathbf{x}_n) = \mathbf{z}_n \qquad \forall n \in [M]. \end{aligned} \tag{38}$$

Moreover, if $D'(\mathbf{z}) \triangleq \phi(D(\mathbf{z}))$, then the autoencoder reconstruction error is preserved:

$$\begin{aligned} \|\mathbf{x}'_n - D'(E'(\mathbf{x}'_n))\|_2 &= \|\phi(\mathbf{x}_n) - \phi(D(E(\mathbf{x}_n)))\|_2 \\ &= \|\mathbf{x}_n - D(E(\mathbf{x}_n))\|_2. \end{aligned} \tag{39}$$

Proof. The equalities follow by direct substitution and the norm-preserving property of ϕ. □

Consequence (worst-case impossibility from Z alone). Proposition 2 shows that the same observed uploaded latent set Z can arise from multiple distinct private datasets that are equally consistent with autoencoder-style reconstruction objectives. Therefore, without additional assumptions linking the coordinates of $\mathcal{X}$ to the semantic structure, no attacker that only observes Z can guarantee recovery of the original private records $\{\mathbf{x}_n\}$.

Why latent-only flipping is weaker than white-box inversion. In standard FL, the server has a white-box copy of the victim reconstruction map $T(\mathbf{x}) = D(E(\mathbf{x}))$ and can directly optimize inputs to reduce the *input-space* objective $r(\mathbf{x}) = \|\mathbf{x} - T(\mathbf{x})\|_2^2$, which tightly couples the attack to the victim model and the true input domain. In GCA, the server does not observe D (or gradients through it) and never sees any $\mathbf{x}_n$; the latent-only objective (36) provides constraints only in latent space and only on the finite set Z. As a result, the attacker must infer an input-space mapping $\tilde{D}$ from latents alone, and the problem is fundamentally underdetermined: many distinct input-space reconstructions are compatible with the same observed latents and can achieve similarly low latent reconstruction loss. This underdetermination prevents a latent-only surrogate from guaranteeing faithful extraction; the comparative strength of the resulting surrogate and white-box attacks is therefore determined empirically.

E Additional Experimental Results and Ablations

All method implementations, shared settings, data partitions, model and K selection, attack procedures, and communication accounting follow Appendix A.

GCA-GMM Component-Count Selection

For each seed and K, GCA accuracy is the maximum over reconstruction-epoch test evaluations. $K^\star$ maximizes the mean of the three seed-level values, with exact ties resolved in favor of smaller K.

Table 3: Selected GCA-GMM component count for each dataset.

	Academic	Adult	Bank	Credit	MAGIC	CIFAR-10	FMNIST
$K^\star$	10	10	10	20	80	20	10

Accuracy by Component Count

Table 4: GCA-GMM accuracy by component count in the IID setting. Values are test accuracy in percent. The two highest unrounded means for each dataset are bold; exact ties are resolved toward smaller K.

Dataset	$K = 10$	$K = 20$	$K = 40$	$K = 80$
Academic	**64.92 ± 0.01**	64.91 ± 0.01	**64.92 ± 0.01**	64.91 ± 0.01
Adult	**66.87 ± 0.02**	**66.87 ± 0.02**	66.87 ± 0.02	66.87 ± 0.02
Bank	**51.18 ± 0.06**	**51.10 ± 0.07**	51.02 ± 0.03	50.95 ± 0.15
Credit	82.31 ± 0.18	**82.40 ± 0.09**	**82.37 ± 0.06**	82.02 ± 0.04
MAGIC	**69.90 ± 0.35**	69.75 ± 0.38	69.80 ± 0.28	**69.97 ± 0.18**
CIFAR-10	**55.39 ± 5.29**	**55.62 ± 5.09**	54.39 ± 5.50	55.29 ± 6.05
FMNIST	**78.61 ± 5.06**	**78.49 ± 6.23**	77.79 ± 5.34	77.30 ± 5.29

Alignment-Phase Weighting Ablation

To isolate the effect of inverse-support weighting on the alignment objective, we compare inverse weighting with uniform weighting, where all nonempty centroid weights are set to $w_k = 1$. Both configurations use the inverse-weighted model's selected $K^\star$ and otherwise share the same data partitions, latent uploads, GMM fitting procedure, reconstruction–alignment schedule, and optimization settings.

Table 5: Peak alignment-epoch accuracy for inverse and uniform weighting at the selected $K^\star$. For each seed, the highest value is selected from the 500 alignment-epoch evaluations over 100 rounds. Values are percentages (mean ± standard deviation); the difference and relative change compare inverse weighting with uniform weighting.

Dataset	$K^\star$	Inverse	Uniform	Difference	Relative change
Academic	10	65.29 ± 0.39	65.12 ± 0.42	+0.18 pp	+0.27%
Adult	10	68.80 ± 0.92	69.20 ± 0.57	-0.40 pp	-0.57%
Bank	10	56.81 ± 2.29	56.48 ± 1.36	+0.34 pp	+0.59%
Credit	20	86.25 ± 0.74	84.89 ± 0.23	+1.36 pp	+1.60%
MAGIC	80	72.93 ± 1.72	74.21 ± 2.35	-1.28 pp	-1.72%
CIFAR-10	20	67.40 ± 10.19	65.15 ± 8.30	+2.25 pp	+3.45%
FMNIST	10	81.97 ± 5.67	81.82 ± 5.83	+0.15 pp	+0.18%

Because centroid weights directly affect the alignment objective, we evaluate their contribution using alignment-epoch accuracy for one fixed client, used consistently across each paired inverse- and uniform-weighting configuration. This diagnostic is separate from the primary all-client reconstruction-accuracy evaluation.

Inverse weighting achieves a higher mean accuracy on five of seven datasets and in 14 of 21 seed-level comparisons. Its unweighted average across the seven datasets is 71.35%, compared with 70.98% for uniform weighting, corresponding to an improvement of 0.37 percentage points. Relative changes range from -1.72% to $+3.45\%$ across all datasets and from $+0.18\%$ to $+3.45\%$ among the five datasets favoring inverse weighting. These results indicate that inverse-support weighting improves peak alignment performance overall, although uniform weighting performs better on Adult and MAGIC.

IID Baseline Comparison

Table 6: Mean ± standard deviation for the IID comparison. The two highest decentralized testing accuracies for each dataset are bold; Single-Client and Centralized (in italics) are excluded from this ranking.

Dataset	GCA	*Single*	FedAvg	FedProx	FedNova	DP-FedAvg	*Centralized*
Academic	**64.92 ± 0.01**	*64.54 ± 0.03*	64.88 ± 0.02	64.90 ± 0.02	64.88 ± 0.02	**65.86 ± 0.27**	*65.95 ± 0.49*
Adult	**66.87 ± 0.02**	*63.25 ± 0.24*	65.84 ± 0.16	66.07 ± 0.03	65.84 ± 0.16	**66.82 ± 0.08**	*69.47 ± 0.28*
Bank	51.18 ± 0.06	*50.74 ± 0.06*	51.65 ± 0.52	51.29 ± 0.14	**51.71 ± 0.50**	**51.92 ± 0.22**	*52.83 ± 0.30*
Credit	**82.40 ± 0.09**	*80.06 ± 0.18*	82.10 ± 0.85	81.39 ± 0.01	**82.22 ± 1.07**	81.00 ± 0.32	*84.08 ± 1.12*
MAGIC	**69.97 ± 0.18**	*68.75 ± 0.06*	69.86 ± 0.73	69.86 ± 0.65	**69.94 ± 0.75**	69.88 ± 0.98	*73.36 ± 0.18*
CIFAR-10	55.62 ± 5.09	*54.09 ± 7.72*	55.86 ± 6.20	**56.56 ± 6.37**	55.94 ± 6.44	**63.71 ± 11.41**	*53.72 ± 9.86*
FMNIST	**78.61 ± 5.06**	*75.02 ± 5.42*	74.33 ± 6.54	74.51 ± 6.99	74.32 ± 6.53	**77.40 ± 2.60**	*75.03 ± 4.50*

GCA improves on Single-Client for all seven dataset means and on FedAvg for five of seven. Against FedProx, FedNova, and DP-FedAvg it records 5/2, 5/2, and 4/3 win/loss counts, respectively. The corresponding mixture of outcomes supports comparable decentralized accuracy rather than uniform dominance.

Clustering-Backend Ablations

To evaluate sensitivity to the clustering backend, we repeat the IID component-count sweep using K-means, soft K-means, and mini-batch K-means. All other data, training, weighting, and reporting settings match the GCA-GMM evaluation.

Table 7: GCA–K-means accuracy by component count in the IID setting. Values are test accuracy in percent (mean ± standard deviation over three seeds). The two highest unrounded means for each dataset are bold; exact ties are resolved toward smaller K.

Dataset	$K = 10$	$K = 20$	$K = 40$	$K = 80$
Academic	64.91 ± 0.01	64.91 ± 0.01	**64.92 ± 0.00**	**64.92 ± 0.00**
Adult	**66.87 ± 0.02**	**66.87 ± 0.02**	66.87 ± 0.02	66.87 ± 0.02
Bank	**51.13 ± 0.15**	**51.12 ± 0.03**	50.91 ± 0.04	50.85 ± 0.15
Credit	**82.56 ± 0.10**	82.47 ± 0.27	**82.48 ± 0.10**	82.42 ± 0.20
MAGIC	**70.07 ± 0.33**	69.93 ± 0.37	69.74 ± 0.35	**69.95 ± 0.28**
CIFAR-10	**55.08 ± 5.82**	55.04 ± 6.36	**55.37 ± 6.53**	55.05 ± 6.27
FMNIST	**79.29 ± 5.05**	**79.29 ± 5.53**	77.85 ± 4.93	77.43 ± 4.94

Table 8: GCA–Soft-K-means accuracy by component count in the IID setting. Values are test accuracy in percent (mean ± standard deviation over three seeds). The two highest unrounded means for each dataset are bold; exact ties are resolved toward smaller K.

Dataset	$K = 10$	$K = 20$	$K = 40$	$K = 80$
Academic	64.91 ± 0.01	64.91 ± 0.02	**64.91 ± 0.01**	**64.91 ± 0.01**
Adult	**66.87 ± 0.02**	**66.87 ± 0.02**	66.87 ± 0.02	66.87 ± 0.02
Bank	**51.16 ± 0.09**	50.97 ± 0.16	**51.06 ± 0.09**	50.86 ± 0.03
Credit	**82.53 ± 0.18**	82.42 ± 0.06	**82.43 ± 0.28**	82.23 ± 0.02
MAGIC	**70.17 ± 0.40**	**69.89 ± 0.24**	69.80 ± 0.36	69.88 ± 0.35
CIFAR-10	**55.24 ± 6.37**	54.96 ± 6.02	**54.97 ± 6.31**	54.51 ± 5.56
FMNIST	**79.38 ± 4.86**	**79.24 ± 5.48**	77.66 ± 5.10	77.26 ± 5.20

Table 9: GCA–MiniBatch-K-means accuracy by component count in the IID setting. Values are test accuracy in percent (mean ± standard deviation over three seeds). The two highest unrounded means for each dataset are bold; exact ties are resolved toward smaller K.

Dataset	$K = 10$	$K = 20$	$K = 40$	$K = 80$
Academic	**64.92 ± 0.01**	64.90 ± 0.02	**64.92 ± 0.03**	64.92 ± 0.01
Adult	**66.87 ± 0.02**	**66.87 ± 0.02**	66.87 ± 0.02	66.87 ± 0.02
Bank	51.07 ± 0.12	**51.16 ± 0.34**	**51.11 ± 0.03**	51.04 ± 0.14
Credit	**82.45 ± 0.19**	**82.43 ± 0.19**	82.37 ± 0.18	82.34 ± 0.07
MAGIC	**70.36 ± 0.43**	**70.40 ± 0.55**	70.23 ± 0.52	69.76 ± 0.36
CIFAR-10	54.77 ± 5.66	55.31 ± 6.28	**55.39 ± 5.42**	**55.52 ± 6.72**
FMNIST	**78.83 ± 4.45**	**78.32 ± 6.18**	77.63 ± 5.04	77.40 ± 4.82

Accuracy varies only modestly across the evaluated component counts and clustering backends. This indicates limited sensitivity to the specific clustering implementation rather than uniform superiority of GMM. We use GMM as the primary backend because its soft responsibilities provide effective support counts directly for inverse-support weighting.

Server-Side Extraction Results

Let $\mathcal{T}_i$ be the complete training shard of the one evaluated client and let $\mathcal{R}_i = \{\hat{\mathbf{x}}_j\}_{j=1}^{100}$ be its 100 reconstructed attack outputs. For every target $\mathbf{x} \in \mathcal{T}_i$, we first select the output with minimum MSE. Cosine similarity is then evaluated against that same selected output, rather than choosing a different output that maximizes cosine. This target-oriented metric measures how well the reconstructed output bank covers the complete client shard. The same reconstructed output may be nearest to multiple training targets, so the metric is not a one-to-one record-recovery measure.

A fixed reference bank $\mathcal{H}_i$ containing 100 normal records excluded from training provides the dataset- and client-specific baseline. Let $\overline{\mathrm{MSE}}(\mathcal{A}, \mathcal{B})$ denote the mean, over records in $\mathcal{A}$, of MSE to the nearest record in $\mathcal{B}$. Let $\overline{\cos}_{\mathrm{MSE}}(\mathcal{A}, \mathcal{B})$ denote mean cosine similarity to those same MSE-selected records. The two reported leakage metrics are

$$\mathrm{NTMSE} = \frac{\overline{\mathrm{MSE}}(\mathcal{T}_i, \mathcal{R}_i)}{\overline{\mathrm{MSE}}(\mathcal{T}_i, \mathcal{H}_i)}, \qquad \Delta\cos = \overline{\cos}_{\mathrm{MSE}}(\mathcal{T}_i, \mathcal{R}_i) - \overline{\cos}_{\mathrm{MSE}}(\mathcal{T}_i, \mathcal{H}_i).$$

NTMSE (normalized target MSE) is a distance ratio, so values above one indicate that reconstructed outputs are farther from training targets than the held-out reference bank. Excess cosine, $\Delta\cos$, is the corresponding similarity difference; negative values mean the reconstructed outputs have lower cosine similarity to the targets than the held-out bank. Consequently, higher NTMSE and lower $\Delta\cos$ indicate less leakage. We average records within the evaluated client and then report the mean and standard deviation over the three dataset seeds.

Table 10: Per-dataset extraction results for one evaluated client per seed (mean ± standard deviation over three seeds). For each training record, the nearest of 100 reconstructed outputs is selected by MSE, and cosine similarity uses that same output. Higher NTMSE and lower excess cosine similarity ($\Delta\cos$) indicate lower target resemblance under the evaluated attack; bold marks the lowest-target-resemblance result per dataset and metric.

Dataset	Method	NTMSE ↑	Δcos ↓
Academic	GCA-GMM	1.838±0.138	-0.563±0.131
Academic	FedAvg	0.921±0.027	-0.007±0.009
Academic	FedProx	0.958±0.117	-0.029±0.019
Academic	FedNova	0.915±0.035	-0.011±0.005
Academic	DP-FedAvg	**276.171±182.022**	**-0.724±0.003**
Adult	GCA-GMM	**2.216±0.140**	**-0.702±0.056**
Adult	FedAvg	1.298±0.045	-0.098±0.006
Adult	FedProx	1.291±0.026	-0.096±0.004
Adult	FedNova	1.265±0.025	-0.092±0.005
Adult	DP-FedAvg	2.060±0.133	-0.563±0.160
Bank	GCA-GMM	**1.584±0.065**	**-0.612±0.019**
Bank	FedAvg	0.935±0.037	0.029±0.011
Bank	FedProx	0.916±0.063	0.028±0.026
Bank	FedNova	0.955±0.065	0.025±0.018
Bank	DP-FedAvg	1.149±0.175	-0.301±0.225
Credit	GCA-GMM	**1.352±0.021**	-0.559±0.055
Credit	FedAvg	1.103±0.017	-0.056±0.005
Credit	FedProx	1.346±0.029	**-0.584±0.009**
Credit	FedNova	1.109±0.020	-0.059±0.011
Credit	DP-FedAvg	1.100±0.019	-0.048±0.010
MAGIC	GCA-GMM	4.931±1.023	-0.493±0.202
MAGIC	FedAvg	1.688±0.087	-0.033±0.009
MAGIC	FedProx	1.877±0.203	-0.047±0.031
MAGIC	FedNova	1.741±0.170	-0.048±0.030
MAGIC	DP-FedAvg	**6.047±0.373**	**-0.919±0.091**
CIFAR-10	GCA-GMM	2.139±0.798	**-0.540±0.169**
CIFAR-10	FedAvg	1.584±0.233	-0.457±0.047
CIFAR-10	FedProx	1.600±0.204	-0.454±0.035
CIFAR-10	FedNova	1.586±0.211	-0.438±0.048
CIFAR-10	DP-FedAvg	**5.502±1.130**	-0.318±0.104
FMNIST	GCA-GMM	**13.940±1.236**	**-1.003±0.101**
FMNIST	FedAvg	5.007±0.404	-0.674±0.019
FMNIST	FedProx	5.570±0.562	-0.733±0.054
FMNIST	FedNova	4.987±0.402	-0.663±0.011
FMNIST	DP-FedAvg	9.573±1.488	-0.647±0.106

GCA-GMM has higher NTMSE than FedAvg, FedProx, and FedNova in all 21 comparisons. Its excess cosine similarity is lower in 20 of 21: all seven datasets against FedAvg and FedNova, and six of seven against FedProx. Against DP-FedAvg, GCA-GMM has higher NTMSE on four datasets and lower excess cosine similarity on five. The two metrics therefore consistently favor GCA-GMM over FedAvg, FedProx, and FedNova, while the comparison with DP-FedAvg is mixed.

Deterministic Communication per Round

Table 11: Inputs to the deterministic communication accounting. C is the client count, U is the total uploaded codes per round, D_z is the flattened latent dimension, and P is the autoencoder parameter count. Payload columns are MiB per round; FL total applies identically to FedAvg, FedProx, FedNova, and DP-FedAvg.

Dataset	C	U	D_z	P	GCA up	FL total
Academic	10	390	16	46,758	0.024	3.567
Adult	20	2,460	16	42,654	0.150	6.508
Bank	20	3,960	16	43,167	0.242	6.587
Credit	50	28,350	16	50,349	1.730	19.207
MAGIC	10	1,410	16	38,037	0.086	2.902
CIFAR-10	20	500	1,024	339,219	1.953	51.761
FMNIST	20	600	784	334,609	1.794	51.057

Table 12: Deterministic total communication per round in MiB. GCA columns include the latent-code uplink and the centroid-and-count broadcast to every client. FL denotes the identical full-model upload-and-broadcast payload of FedAvg, FedProx, FedNova, and DP-FedAvg.

Dataset	GCA $K = 10$	GCA $K = 20$	GCA $K = 40$	GCA $K = 80$	FL
Academic	0.030	0.037	0.050	0.076	3.567
Adult	0.163	0.176	0.202	0.254	6.508
Bank	0.255	0.268	0.294	0.345	6.587
Credit	1.763	1.795	1.860	1.990	19.207
MAGIC	0.093	0.099	0.112	0.138	2.902
CIFAR-10	2.735	3.517	5.081	8.209	51.761
FMNIST	2.393	2.992	4.190	6.586	51.057

For all evaluated datasets and all four tested component counts, GCA transmits less data per round than the parameter-sharing baselines. Across the tabular datasets, the reduction ranges from 89.64% (Credit, $K = 80$) to 99.15% (Academic, $K = 10$). With the two-downsampling vision architecture, CIFAR-10 uses 2.735–8.209 MiB per GCA round versus 51.761 MiB for FL, and Fashion-MNIST uses 2.393–6.586 MiB versus 51.057 MiB. Even at $K = 80$, these are reductions of 84.14% and 87.10%, respectively.

Non-IID Results

The non-IID experiments introduce both feature-distribution and sample-count skew. Samples are grouped using their feature descriptors, and a Dirichlet distribution with $\alpha = 0.1$ assigns each feature group concentrated client preferences. Long-tail client quotas additionally impose a target 10:1 maximum-to-minimum sample-count ratio. Clients therefore observe different feature distributions and unequal quantities of normal training data; the complete partition protocol is given in Appendix A.

Table 13: GCA-GMM K sweep in the non-IID setting. Values are test accuracy in percent.

Dataset	$K = 10$	$K = 20$	$K = 40$	$K = 80$
Academic	63.36 ± 0.76	63.36 ± 0.76	63.36 ± 0.76	63.36 ± 0.76
Adult	43.41 ± 2.69	43.41 ± 2.69	43.41 ± 2.69	43.41 ± 2.69
Bank	50.46 ± 0.23	50.46 ± 0.23	50.46 ± 0.23	50.46 ± 0.23
Credit	62.33 ± 0.76	62.33 ± 0.76	62.33 ± 0.76	62.33 ± 0.76
MAGIC	61.75 ± 0.58	61.75 ± 0.58	61.75 ± 0.58	61.75 ± 0.58

Table 14: Selected GCA-GMM ($K^\star = 10$) and FL baselines in the non-IID setting. Values are test accuracy in percent.

Dataset	GCA	FedAvg	FedProx	FedNova	DP-FedAvg
Academic	63.36 ± 0.76	62.24 ± 1.20	62.26 ± 1.20	62.23 ± 1.17	63.88 ± 0.40
Adult	43.41 ± 2.69	38.88 ± 0.87	39.26 ± 1.51	38.88 ± 0.87	39.66 ± 1.26
Bank	50.46 ± 0.23	50.39 ± 0.28	50.42 ± 0.25	50.40 ± 0.29	50.47 ± 0.13
Credit	62.33 ± 0.76	61.19 ± 0.89	60.96 ± 0.74	61.19 ± 0.89	61.19 ± 0.89
MAGIC	61.75 ± 0.58	62.39 ± 0.45	62.45 ± 0.69	61.68 ± 1.03	59.01 ± 1.71

All tested K values attain the same reported reconstruction-epoch peak, so the stated tie rule selects $K^\star = 10$. Compared with FedAvg, GCA improves Academic, Adult, Bank, and Credit and trails on MAGIC, with per-dataset relative changes from -1.03% to 11.66%. The reported 2.26% is the relative difference between the five-dataset macro-averages:

$$\frac{\overline{A}_{\mathrm{GCA}} - \overline{A}_{\mathrm{FedAvg}}}{\overline{A}_{\mathrm{FedAvg}}} \times 100.$$